arXiv:2506.10947v2 [cs.AI] 25 Feb 2026

Spurious Rewards: Rethinking Training Signals in RLVR

Rulin Shao [*1] **Shuyue Stella Li** [*1] **Rui Xin** [*1] **Scott Geng** [*1] **Yiping Wang** [1] **Sewoong Oh** [1] **Simon Shaolei Du** [1]
Nathan Lambert [2] **Sewon Min** [3] **Ranjay Krishna** [1 2] **Yulia Tsvetkov** [1] **Hannaneh Hajishirzi** [1 2]
Pang Wei Koh [1 2] **Luke Zettlemoyer** [1]

Spurious_Rewards

Abstract

We show that reinforcement learning with verifiable rewards (RLVR) can elicit strong mathematical reasoning in certain language models even with *spurious rewards* that have little, no, or outright negative correlation with the correct answer. For example, RLVR training with GRPO improves MATH-500 performance for Qwen2.5-Math-7B in absolute points by 21.4% using randomly assigned rewards, nearly matching the 29.1% gained with ground truth rewards. To explain this counterintuitive observation, we show that GRPO exhibits a *clipping bias* arising from the clip term, which can amplify high-prior behaviors learned during pre-training even without informative rewards. As a case study, we identify one such high-prior behavior for Qwen2.5-Math models, which we term *code reasoning*—reasoning in code without actual code execution; code reasoning frequency increases from 65% to over 90% with spurious rewards. However, the presence of such amplifiable behaviors is highly model-dependent. In practice, spurious rewards that are effective for Qwen models often fail to produce gains for other model families, such as Llama3 or OLMo2. Our results highlight the importance of validating RL methods across diverse models rather than relying on a single de facto choice: large performance gains can arise on Qwen models even from random rewards that do not reflect genuine capability improvements.

1. Introduction

Reinforcement learning with verifiable rewards (RLVR) is highly effective in improving language model reasoning (Lambert et al., 2024; Guo et al., 2025; Zeng et al., 2025; Luo et al., 2025), but the mechanisms underlying these gains remain poorly understood. In this work, we show counterintuitively that RLVR can yield substantial performance improvements on math reasoning even under a family of *spurious rewards* that carry little to no task-relevant signal and that this effect depends critically on the base model. For example, on the Qwen2.5-Math models (Yang et al., 2024a;b), which are widely used in the RLVR literature, randomly assigned rewards yield a 21.4% absolute accuracy gain on MATH-500, compared to a 29.1% gain from ground-truth rewards (§2). We observe similar trends on more challenging math benchmarks, including AMC and AIME. In contrast, for other model families, including Qwen2.5 (Yang et al., 2024b), OLMo2 (OLMo et al., 2024), and Llama3 (Dubey et al., 2024), training with spurious rewards yields minimal improvement or even performance degradation (§3). This divergence in outcomes suggests that pretraining priors, different in each model, strongly shape RL training dynamics.

To further investigate the underlying mechanism contributing to such spurious improvements, we unpack the GRPO training objective (Shao et al., 2024) and find that the clipping function systematically amplify tokens with high prior probabilities in the base model (§4). To help explain this discrepancy, we also analyze what reasoning patterns that RLVR is learning to favor in these cases (§5). In particular, we find a majority of Qwen2.5-Math-7B answers on MATH-500 contain reasoning chains expressed in Python—a behavior we call *code reasoning*—despite having no access to code execution. Code reasoning is highly predictive of overall performance; answers with it have an accuracy of 60.9%, much higher than without (28.0% accuracy). Code reasoning also correlates with MATH-500 accuracy over the course of RLVR training. Both metrics increase consistently during training with any spurious reward, leading to roughly 90% or higher code frequency after training. Be-

[*]Equal contribution [1]University of Washington, Seattle, WA, USA [2]Allen Institute for Artificial Intelligence, Seattle, WA, USA [3]University of California, Berkeley, Berkeley, CA, USA. Correspondence to: <{rulins,stelli,rx31,sgeng}@cs.washington.edu>.

Preprint. February 26, 2026.

01.01 摘要·问题

奖励与正确性很弱相关、无关甚至负相关时，RLVR是否仍能提高数学成绩？本文把奖励质量逐步削弱来检验。

01.02 摘要·结果

Qwen2.5-Math-7B随机奖励提升21.4个百分点，真实奖励提升29.1个百分点。这里是准确率绝对差，不是相对增长率。

01.03 摘要·解释

作者提出：GRPO裁剪带来的优化偏置可放大预训练已有行为；代码推理是可观察案例，而非对全部能力变化的完整解释。

01.04 摘要·边界

收益高度依赖基模。单一Qwen模型上的显著涨分，不足以证明奖励含有有效任务监督或方法可跨模型迁移。

01.05 引言·动机

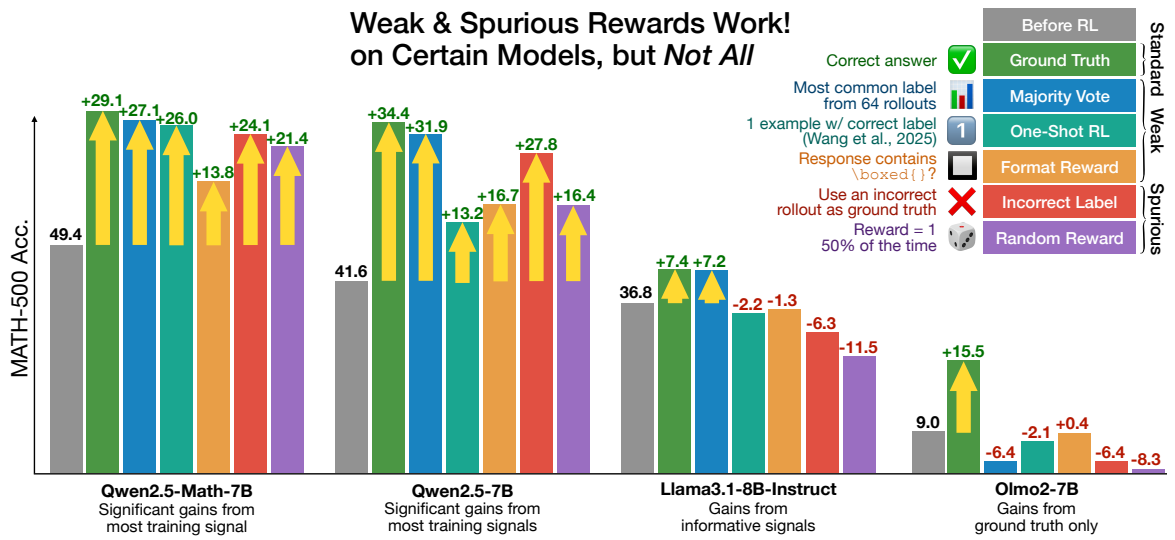
RLVR有效，但涨分的来源不清楚。作者通过替换奖励，分离正确性监督、优化算法与基模先验的作用。

01.06 引言·路线

先分析裁剪梯度，再追踪训练中的推理策略。代码及其所谓运行结果均由模型生成，没有真正调用解释器。

01.07 引言·观察

代码输出与正确率相关，训练中二者也同步上升；相关性尚不能单独确立因果。此处无代码准确率28.0%，与表1的35.0%不一致，引用数值须核对口径。



02.01 *Figure 1.* MATH-500 accuracy after 300 steps of RLVR on various training signals. We show that even "spurious rewards" (e.g., rewarding *incorrect* labels or with completely random rewards) can yield strong MATH-500 gains on Qwen models. Notably, these reward signals do not work for other models like Llama3.1-8B-Instruct and OLMo2-7B, which have different reasoning priors.

02.02 sides code reasoning, we find that lexical repetitions show similar but less substantial correlation with MATH performance in Qwen2.5-Math-7B models. Based on this observation, we further hypothesize that intervening with other methods that increase any beneficial behavior should similarly increase test performance. Our experiments validate this hypothesis: we design prompt-based and RL-based elicitation methods to increase code reasoning and lexical repetition; all such methods significantly increase Qwen2.5-Math-7B's performance. We show that these results are robust to prompt variations, despite the initial model performance being sensitive to prompts in sometimes unexpected ways (Appendix I).

02.03 Our findings not only open new questions but also have practical implications. We should generally be more aware that reasoning patterns instilled during pretraining heavily impact the behavior of downstream RLVR training. Qwen models, with open weights and high performance on reasoning tasks, have become the de facto choice for RLVR research in the open-source community—a range of recent research on RLVR drew conclusions on Qwen2.5-Math-7B-centric experiments (Zuo et al., 2025; Zhao et al., 2025b; Wang et al., 2024; Xie et al., 2025; Hu et al., 2025b; Zhang et al., 2025; Shafayat et al., 2025; Prabhudesai et al., 2025; Gao et al., 2025; Wang et al., 2025a). However, we show that it is easy to get significant performance improvements on Qwen models even with completely spurious reward signals. Thus, we suggest that future RLVR research should be confirmed on other models and using spurious rewards as dummy baselines. We emphasize that spurious rewards are proposed purely for analytical purposes and should not be interpreted as a recommended approach for developing true model capabilities.

2. Spurious Rewards Yield Significant RLVR Gains

02.04 We design a progression of reward functions to replace the standard ground-truth reward: *weak rewards* (majority vote reward and format reward) and *spurious rewards* (random reward and incorrect reward). Remarkably, we find that all weak and spurious rewards suffice for RLVR to significantly improve the math performance of Qwen2.5-Math, a popular starting point for RLVR training.

2.1. Experimental Setup

02.05 Following recent RLVR work (Wang et al., 2025b; Zuo et al., 2025; Zeng et al., 2025), we use GRPO (Guo et al., 2025) to finetune Qwen2.5-Math models (Yang et al., 2024a). The standard RLVR approach uses a dataset of questions paired with ground truth labels. During training, model rollouts are given a binary (0-1) reward based on whether the generated answer is verifiably correct. We replace this ground truth-based reward with a variety of increasingly spurious binary 0-1 reward functions that do not require access to ground truth labels. We design these alternative rewards to investigate the limits of how little supervision is needed for effective RLVR training. We train on DeepScaleR data (Luo et al., 2025) with our rewards; all other experimental details are kept constant.

02.06 In the main paper, we evaluate performance as pass@1 and average@8 accuracy on two standard math reasoning benchmarks: MATH-500 (Hendrycks et al., 2021) and AMC (Li et al., 2024), respectively. See Appendix D for additional results on AIME 2024 and 2025. Following the default evaluation setup in the popular RL framework OpenRLHF (Hu

02.01 图1·全篇总览

读柱顶的绝对增益，再看基线：同样的虚假奖励在Qwen上常有收益，在OLMo/Llama上可能无效或有害。基模是关键实验变量。

02.02 引言续·干预

除代码策略，还考察减少词汇重复。通过提示或奖励主动诱导有益行为，检验"已有行为被激发"的解释。

02.03 引言续·研究建议

未来应加入随机、格式等奖励作为对照，并覆盖不同预训练分布的模型。虚假奖励是分析工具，不是推荐的训练方案。

02.04 第2节·设计

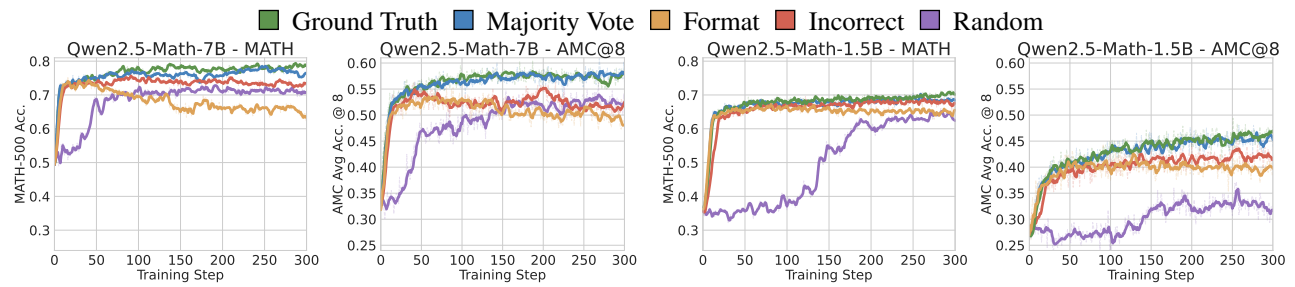
从真实奖励到多数投票、格式，再到错误标签和随机奖励，逐步减少与正确答案相关的监督信息。

02.05 设置·控制变量

用GRPO在DeepScaleR上微调；以二值奖励替换真实正确性，其余配置尽量保持一致。错误标签子集的筛选仍需要真实标签作分析。

02.06 设置·指标

MATH-500用pass@1；AMC用average@8，即8次采样正确率的平均值，不是"8次中至少一次正确"的pass@8。



(a) Qwen2.5-Math-7B *(b)* Qwen2.5-Math-1.5B

03.01 *Figure 2.* Model performance on MATH and AMC with varied training rewards smoothed over window size of 10 (dotted lines are unsmoothed values). We report pass@1 for MATH and average@8 for AMC. Both Qwen2.5-Math-7B and Qwen2.5-Math-1.5B significantly improve after RLVR on a range of reward signals from meaningful to spurious. We note that the random reward converges slower than the other spurious rewards, but the fact that it leads to significant gains at all is surprising.

03.02 et al., 2025a), we use the default chat template for Qwen2.5-Math and all instruct models and chat template provided by Olmo (OLMo et al., 2024) for other base models in our main experiments. Additional analysis on the effect of different prompts can be found in Appendix I, where we show Qwen2.5-Math-7B is very sensitive to prompts—even a task-irrelevant prompt (which we name *spurious prompt*) can sometimes result in high initial performance. See Appendix A for full details of our training and evaluation setup.

2.2. Standard to Weak to Spurious Rewards

We consider the following rewards:

03.03 1. **Ground Truth Rewards:** To establish a baseline, we consider the standard RLVR approach (Lambert et al., 2024) of using ground truth labels to reward responses with verifiably correct answers. This setting serves as an upper bound for reward supervision quality.

03.04 2. **Majority Vote Rewards:** Instead of using ground truth labels for computing rewards, we use the model prior to RLVR training to pseudo-label the training set by selecting the majority answer from 64 sampled responses per prompt. These (potentially wrong) labels are then used to reward responses during standard online RLVR training.

03.05 3. **Format Rewards:** We further weaken the reward signal to disregard the responses' mathematical correctness altogether. We instead heuristically reward all responses containing at least one non-empty `\boxed{}` expression, regardless of the correctness of the enclosed answer. Including `\boxed{}` is specified in Qwen2.5-Math's system prompt; this reward incentivizes some degree of prompt following.

4. **Random Rewards:** We study whether providing *no* guidance in the rewarding process is sufficient to provide meaningful math performance gains. To do so, we assign rewards randomly. Given a fixed probability hyperparameter γ, all responses receive a reward of 1 with chance indicated by the parameter, and receive 0 otherwise. In our main experiments, we present $\gamma = 0.5$; in Appendix B, we show that using $\gamma \in \{0.001, 0.3, 0.7\}$ obtains similar improvements with varying convergence speed, and verify that $\gamma = 0$ results in no change as expected analytically (with $\gamma = 0$, loss is constant, and all gradients are zero).

03.07 5. **(Majority-voted) Incorrect Rewards:** We furthermore deliberately provide incorrect supervision and reward only incorrect answers. We first label all training data using majority voting and select the subset with incorrect labels for training, obtaining incorrect labels that are still probable outputs of the models. During training, we reward responses whose answers verifiably match these incorrect labels. This design disentangles whether majority vote rewards work because labels are more likely correct or because they represent high-probability model outputs. We reward specific incorrect answers rather than any incorrect answer to maintain comparable reward sparsity with our other conditions.

Note: We emphasize that spurious rewards—particularly random and incorrect rewards—are proposed purely for analytical purposes and should not be interpreted as a recommended approach for developing true model capabilities.

03.08 Figure 2 presents the performance of Qwen2.5-Math models after RLVR training with each reward function. Overall, all reward functions, even pathologically designed ones, lead to significant improvements in math performance within the first 50 steps across all benchmarks compared to the untuned baseline. One exception is that Qwen2.5-Math-1.5B sees 03.06 gains with random rewards much slower (after 100 steps) and less so on AMC (only 4.9%). Remarkably, performance gains from spurious rewards are often within a few points of the gain from RLVR with ground truth labels. For example, training with incorrect label reward yields 24.1% gains over Qwen2.5-Math-7B on MATH-500, compared to

03.01 图2·主要结果

两种尺寸Qwen-Math在多种奖励下都涨分；随机奖励收敛更慢。实线为10步平滑，虚线为原始波动，比较时要兼顾速度与最终水平。

03.02 设置续·提示模板

不同模型用对应聊天模板；Qwen-Math对提示很敏感，甚至无关提示也可改变初始成绩。因此提示必须纳入对照。

03.03 奖励1·真实答案

输出答案与标准答案等价才给1。它提供最直接的正确性监督，是本文比较的强监督基线。

03.04 奖励2·固定伪标签

RL之前用基模对每题采样64次，取多数答案，之后固定使用；并非随训练在线更新的多数投票。伪标签可能错误。

03.05 奖励3·格式

只检查至少一个非空的boxed表达式，不检查数学正确性。它能强化答案格式与提示遵循，但并非完全无结构的噪声。

03.06 奖励4·随机

每条回答独立按Bernoulli(γ)抽0或1，主实验γ=0.5；奖励与该回答内容无关。γ=0时组内优势恒为0，不更新。

03.07 奖励5·具体错误标签

先多数投票，再筛出伪标签确实错误的题；只有匹配那个特定错误答案才得奖，不是任意错误都得奖。这样保留较高概率答案与相近奖励稀疏性。

03.08 结果·读数

多数奖励在早期已有明显收益；1.5B随机奖励较慢且AMC收益较小。这种成功有模型尺寸、任务与训练步数条件。

蓝色数字对应段落 / 图表；术语在首次出现处批注

a 29.1% gain from RLVR with ground truth answers. Even random rewards—which by design provide pure noise in the rewarding process —still produce a 21.4% performance boost. We observe similar trends in AMC, where training on format, incorrect, or random rewards yields a gain of 13.8%, 24.1%, or 21.4%, respectively, approaching the $\sim$ 27%–29% improvement gained from training on majority voted and ground truth labels.

In Appendix D, we supplement results on AIME24 and AIME25. On AIME2024, format reward (+10.3%) approaches ground truth rewards (+15.3%), and spurious rewards (incorrect & random) still lead to high performance gains of 10.2% and 10.2% respectively on Qwen2.5-Math-7B. Ground truth labels show a clear advantage compared to other rewards on AIME2025, which contains questions written after the knowledge cutoff of all models we consider. Nonetheless, other rewards still lead to a -0.4% to 4.5% gain in performance.

Our findings with these simple rewards provide additional evidence for a nascent hypothesis in the literature: that RLVR, at least at the compute scales of open-source post-training pipelines (Lambert et al., 2024), does not teach models new reasoning capabilities, but instead triggers latent ones already present in the base model (Wang et al., 2025b; Liu et al., 2025; Gandhi et al., 2025; Yue et al., 2025; AI et al., 2025; Choshen et al., 2020). Whereas prior work has hinted at this effect using limited-quantity ground-truth labels (Wang et al., 2024) or noisy labels (Zuo et al., 2025), our results push this idea to its limit: we show that even outright incorrect rewards or information-free rewards (i.e., random) can elicit performance gains in Qwen2.5-Math models. In the remainder of our paper, we show that this elicitation effect is model-dependent (§3), and trace the specific properties of Qwen2.5-Math models that could enable spurious rewards to induce this elicitation (§5).

3. (Lack of) Generalization to Other Models

Inspired by the unexpected effectiveness of spurious reward signals in improving the performance of Qwen2.5-Math models, we study whether these rewards generalize to training other models. We extend beyond the math-specialized Qwen2.5-Math models to include general-purpose variants (Qwen2.5-7B, Qwen2.5-1.5B (Yang et al., 2024b)), and two additional model families: (a) the widely-used Llama3.1-8B(-Instruct) and Llama3.2-3B(-Instruct) (Dubey et al., 2024), and (b) OLMo2-7B and OLMo2-7B-SFT (OLMo et al., 2024). OLMo2-7B-SFT is instruction-tuned from OLMo2-7B; we include both to better understand the impact of SFT training on RLVR training. Moreover, we are optimistic that OLMo's open training data will enable future works to better study the origins of any reasoning behaviors (or lack thereof) we later observe. We train these 8 additional models with the same setup and rewards as in Section 2. We report performance on MATH-500 in the main text of the paper and AMC in Appendix C where the same trends are observed. The AIME 2024 and 2025 benchmarks show similar but noisy trends (Appendix D).

Spurious rewards can benefit Qwen2.5 models, but rarely help non-Qwen models. Figure 3 shows two trends. First, models within a family behave similarly: across Qwen2.5, all non-random rewards (even spurious incorrect) improve MATH-500, whereas OLMo stays flat under spurious rewards and gains mainly with ground-truth rewards. We conjecture this family-level consistency reflects shared pretraining distributions that induce similar pre-RL behaviors. Second, smaller models benefit less from spurious (e.g., random) rewards; we conjecture larger models preserve more pretraining priors that these rewards amplify. Section 5 further analyzes how pre-existing behaviors shape RLVR outcomes. Importantly, these reward signals do not reliably transfer across families. Although spurious incorrect and other weak rewards consistently improve Qwen, each weak or spurious reward fails to help at least one other model and can be flat or even harmful. Overall, Qwen appears unusually robust to reward strength. Appendix J further shows that models already RL post-trained see minimal gains under nearly all rewards.

Practical warning ⚠️: Proposed RLVR reward signals should be tested on diverse models! Many recent methods on RLVR for reasoning draw their conclusions primarily or exclusively from gains shown on Qwen models (non-exhaustively, (Zuo et al., 2025; Zhao et al., 2025b; Wang et al., 2024; Xie et al., 2025; Hu et al., 2025b; Zhang et al., 2025)). As a case study, we experimented with recent work on (1) test time training (Zuo et al., 2025) and (2) one-shot RL (Wang et al., 2025b) (setup details in Appendix A.7). We find these methods exhibit a pattern similar to our results above: the proposed training signals yield strong improvements in the Qwen2.5-Math or Qwen2.5 models, matching the performance of the ground truth rewards (Figure 15 in Appendix E). However, these same signals often fail to yield performance gains on other model families. Our findings suggest that existing Qwen-centric RLVR research should possibly be further validated on non-Qwen models.

4. Understanding Training Signals from Random Rewards in GRPO

Prior work has shown that RLVR using majority-voted labels or format rewards can improve model performance in GRPO (Zuo et al., 2025; Zhao et al., 2025b; Wang et al., 2025b). As a result, the effectiveness of majority-voted

04.01 结果续·核心比较

MATH-500：错误标签提升24.1点，随机提升21.4点，真实奖励29.1点。紧随其后的AMC数字疑似复用了MATH结果，宜结合图2核对。

04.02 更难测试集

AIME24仍有明显收益；AIME25上真实奖励优势更清楚，其他奖励增益较小。新年份题目提供更严格的泛化检验。

04.03 解释·能力激发假说

结果支持“在本研究计算规模下，RL可能主要激发已有能力”。这不是严格证明RL永远不能学到新能力。

04.04 第3节·跨模型检验

加入通用Qwen、Llama及OLMo的基模和指令模型，考察收益是否超出数学专用模型；SFT前后版本用于观察后训练影响。

04.05 发现·家族差异

通用Qwen对许多弱/虚假奖励仍有收益；其他家族通常需要较可信的监督。作者把这种差异联系到预训练行为先验，尚属解释性假说。

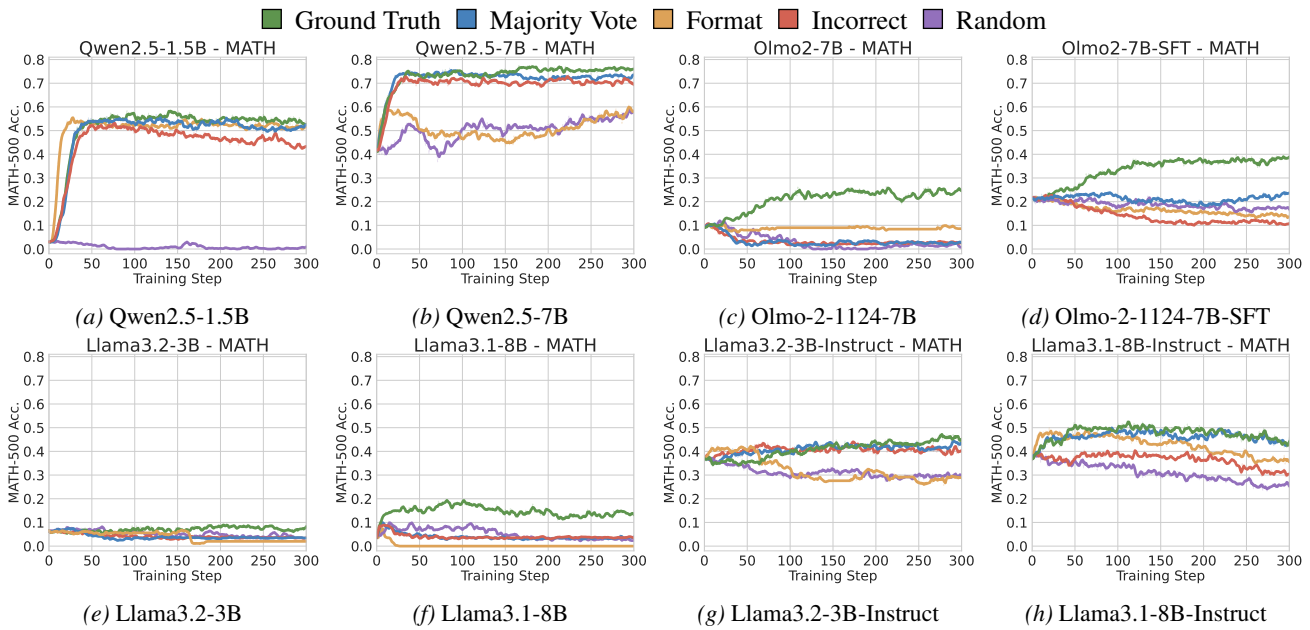
04.06 方法学·外推边界

复测TTRL与单样本RL也出现相似家族差异。由于未充分跨模型调参，且单样本实验去除了熵项，结论应限定于本篇配置。

04.07 第4节·机制问题

多数票和格式奖励尚含结构信息；随机奖励完全不看回答，因此它能涨分尤其需要从优化过程解释。

蓝色数字对应段落 / 图表；术语在首次出现处批注



(a) Qwen2.5-1.5B *(b)* Qwen2.5-7B *(c)* Olmo-2-1124-7B *(d)* Olmo-2-1124-7B-SFT

(e) Llama3.2-3B *(f)* Llama3.1-8B *(g)* Llama3.2-3B-Instruct *(h)* Llama3.1-8B-Instruct

05.01 *Figure 3.* Varying rewards across additional model classes on MATH-500. Spurious rewards remain effective on general-purpose Qwen2.5 models but generally fail to yield any gains on other model families. The performance improvements on non-Qwen2.5 models are substantially smaller compared to those observed in the Qwen2.5 family. Similar trends are observed on AMC in Appendix C.

(even when incorrect[1]) rewards and format rewards is less surprising. However, the effectiveness of random rewards remains puzzling, as they are constructed to contain no information. In this section, we present our hypotheses on how random rewards induce meaningful training signals in GRPO and validate them through ablation studies.

05.03 **No training signal from random rewards when clipping is removed from GRPO.** We analyze GRPO with the clipping term removed and show that random rewards alone do not yield effective training. The clipping term appears in the GRPO objective[2]:

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_{\text{old}}(\cdot|x)} \left[\sum_{t=1}^{|y|} \min\Big(\rho_t(y;\theta)\,\hat{A}(x,y), \; \text{clip}\big(\rho_t(y;\theta), 1-\epsilon_c, 1+\epsilon_c\big)\hat{A}(x,y)\Big)\right], \tag{1}$$

05.04 where x denotes the input, y the generated sequence, π_{old} the behavior policy, $\rho_t(y;\theta) = \pi_\theta(y_t \mid x, y_{<t})/\pi_{\text{old}}(y_t \mid x, y_{<t})$ the importance ratio at step t (corresponding to the t-th generated token), and ϵ_c the clipping threshold. The advantage $\hat{A}(x,y)$ is computed from the reward $r(x,y)$ by centering it within each prompt, $\hat{A}(x,y) = r(x,y) - \mathbb{E}_{y'\sim\pi_{\text{old}}(\cdot|x)}[r(x,y')]$. When clipping (the red term in Eq. 1) is removed and rewards are sampled independently of the responses, the expected training objective is zero, yielding no meaningful learning signal. To demonstrate this, we consider three separate **no-clipping variants** of GRPO: disabling clipping in the implementation, increasing the mini-batch size to match the rollout size, or reducing the rollout size so that only a single gradient update is performed per rollout.[3] As shown in the first three panels of Figure 4 (no-clipping variants), random rewards fail to yield consistent performance. In contrast, enabling clipping (bottom-right) leads to stable and consistent improvements.

05.06 **GRPO clipping bias can induce random reward training signals.** We analyze the effect of clipping in GRPO under random rewards, with the full derivation provided in Appendix B. At a high level, clipping introduces a bias in the gradient update that depends on how the current policy's token probabilities deviate from those of the behavior policy.

05.07 Factoring out reward-independent scalar terms, the expected gradient induced by random rewards can be written (up to a positive constant) as

$$\mathbb{E}[\nabla_\theta J(\theta)] \propto \mathbb{E}_{x,y}\left[\begin{cases} \nabla_\theta \log \pi_{\theta,x}(y_t), & R_\theta < 1-\epsilon_c, \\ 0, & |R_\theta - 1| \le \epsilon_c, \\ -\nabla_\theta \log \pi_{\theta,x}(y_t), & R_\theta > 1+\epsilon_c, \end{cases}\right], \tag{2}$$

where $R_\theta = \pi_{\theta,x}(y_t)/\pi_{\text{old},x}(y_t)$. This form shows that clipping increases the probability of tokens that are already

[1] We hypothesize that incorrect rewards provide training signals because many remain close to ground truth and because they implicitly act as format rewards, incentivizing extractable answers.

[2] We omit the KL regularization term, which is disabled in our experiments.

[3] The latter two variants enforce $\pi_\theta = \pi_{\text{old}}$ during each rollout, yielding $\rho_t = 1$ and thus eliminating the effect of clipping by construction.

05.01 图3·泛化对照

按模型横向比较五种奖励。Qwen家族较容易从弱信号受益，其他家族收益较少，显示单一基模基准可能高估方法普适性。

05.02 机制·研究对象

接下来的推导针对随机奖励，不能直接把“奖励独立于回答”的前提套到格式、多数票或错误标签奖励上。

05.03 推导·无裁剪基线

独立随机奖励的中心化优势期望为0；在理想独立假设下，无裁剪策略梯度的总体期望也为0。有限批次仍有噪声梯度。

05.04 公式1·符号

旧策略π_old生成样本，当前策略π_θ更新；ρ为同一上下文下token概率之比，优势衡量回答在组内的相对奖励。本文关闭KL项。

05.05 消融·三种方式

直接删裁剪，或调整批量让每批采样只更新一次。后两种让计算梯度时ρ=1，同时减少更新次数，不能当作优化预算完全相同的对照。

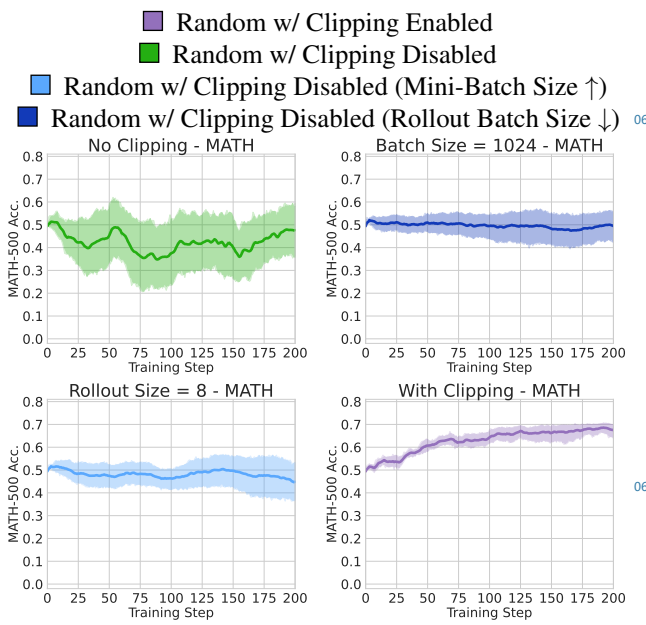
05.06 推导·有裁剪

裁剪使正负优势对应的梯度不再对称抵消。比值落在下界外时偏向增加该token概率，上界外时偏向减少，区间内期望抵消。

05.07 公式2·精读

主文把结果写成score梯度方向；附录精确式含μ□ρ，且□ρ=ρ□logπ。ρ随样本变化，研究总体期望时不能把它当统一常数丢掉。

蓝色数字对应段落 / 图表；术语在首次出现处批注



06.01 *Figure 4.* Average performance over multiple random seeds when ablating the clipping term in GRPO with random rewards for Qwen2.5-Math-7B. The first three panels correspond to *no-clipping variants* (see text), while the last panel enables clipping. Random rewards yield consistent improvements only when clipping is present.

likely under the policy, while suppressing overly large deviations, even when rewards are uninformative. We illustrate this effect with the following example:

06.02 **Example.** Let clipping threshold $\epsilon_c = 0.2$ and consider two tokens with different probabilities under the policy. The same relative clipping threshold therefore induces asymmetric absolute clipping ranges within $[0, 1]$ for low- and high-probability tokens. If $\pi_{\text{old},x}(y_t) = 0.85$ (frequently sampled), the policy probability would need to exceed $\pi_{\text{old},x}(y_t) \cdot (1 + \epsilon_c) = 1.02$ to trigger upper clipping. Since $\pi_{\theta,x}(y_t) \leq 1$, this threshold can never be reached. Thus, this token receives nonnegative gradient bias, reinforcing its high probability. In contrast, if $\pi_{\text{old},x}(y_t) = 0.02$ (rarely sampled), upper clipping is triggered when $\pi_{\theta,x}(y_t) > 0.024$. Small increases in $\pi_{\theta,x}(y_t)$ can easily exceed this threshold, causing this token to receive negative gradient bias that suppresses its probability. Thus, clipping bias asymmetrically suppresses low-probability tokens and reinforces high-probability ones.

06.04 In Appendix B, we show that this effect persists across different random reward distributions, indicating its stability. Our findings echo the intuition of Yu et al. (2025), who show that clipping bias reduces exploration and increases exploitation in RLVR with ground-truth labels.

5. Case Study: Code Reasoning as a Representative Pre-Existing Behavior

06.05 Previously, we showed that different models can exhibit substantially different outcomes when trained with the same weak or spurious reward signals. Here, we propose a general hypothesis for these discrepancies and examine it through a focused case study of a representative pre-existing behavior observed in Qwen models. Our central hypothesis is that differences in RLVR outcomes stem from differences in the reasoning strategies acquired during pretraining. Some strategies may be readily amplified by RLVR, while others may be difficult to elicit or absent altogether, leading to divergent training dynamics across model families.

06.06 We study one such strategy—using code to support mathematical reasoning—which Qwen-Math models employ effectively, while other model families do so to a lesser extent (§5.1). By tracing the prevalence of code reasoning over the course of RLVR training, we find evidence consistent with this hypothesis (§5.2). We note that we use code reasoning as a controlled and observable example to study how RLVR interacts with pre-existing internal strategies, rather than as an exhaustive characterization of model behavior.

5.1. Different Models Exhibit Pre-existing Reasoning Strategy Discrepancies

06.07 We analyze reasoning traces from different models on MATH-500 and observe clear discrepancies in pre-existing strategies. Qwen2.5-Math-7B frequently generates Python code in its reasoning (65.0% of responses), despite lacking code execution; we refer to this behavior as *code reasoning*. Crucially, code reasoning in Qwen2.5-Math-7B is strongly correlated with correctness. As shown in Table 1, responses containing code achieve substantially higher accuracy than those without. Qwen2.5-Math-1.5B exhibits similar behavior, while other models do not. We categorize these models as either *No-Code* (e.g., Llama, Qwen2.5-1.5B, OLMo2-7B), which do not generate code, or *Bad-Code* (e.g., OLMo2-7B-SFT, Qwen2.5-7B), which frequently generate code but with degraded performance. Thus, effective code reasoning appears to be a distinctive pre-existing capability of the Qwen2.5-Math prior to RLVR.

06.08 We provide further analysis that hints at the origins of code reasoning of Qwen2.5-Math-7B models in Appendix K. Note that code reasoning is not used as a complete explanation: other behaviors can also be elicited easily and often correlate with performance—we briefly discuss another such behavior, generation without repetition, in Appendix G. In addition, we find the initial performance of Qwen2.5-Math-7B is sensitive to the prompts used for evaluation; we supplement a discussion on this impact in Appendix I.

06.01 图4·证据

多随机种子的平均曲线：开启裁剪时收益较稳定；关闭后不再稳定提高。不是说每个无裁剪运行都绝无涨分，附录有高方差个例。

06.02 概率例子·高概率

ε=0.2，旧概率0.85，则上界1.02超过概率可取范围，无法触发上侧裁剪。这个边界例子说明相对裁剪的绝对范围不对称。

06.03 概率例子·低概率

旧概率0.02时上界仅0.024，下界0.016；相同相对幅度对应更窄绝对区间。它说明局部偏置，不能保证神经网络每次更新都提高所有高概率token。

06.04 机制边界

刚采样后π_θ=π_old、ρ=1，此时没有上述区间外偏置；同一批样本的后续小批更新使两策略偏离，裁剪才可能生效。

06.05 第5节·假说

相同训练信号的效果取决于基模已有的推理策略：有用且容易被强化的策略越多，弱监督越可能获益。

06.06 案例定位

选代码推理作为可观察策略，追踪频率并施加干预。论文没有声称它能穷尽所有模型行为或所有涨分原因。

06.07 5.1·行为分组

Qwen-Math常写代码且代码回答更准；No-Code几乎不写；Bad-Code虽常写却不如自然语言准确。关键在于策略质量，而非“是否写代码”。

06.08 补充因素

附录另研究重复输出、提示敏感性与代码行为来源。这些因素也可能改变可解析率和成绩，需要与数学能力增长区分。

蓝色数字对应段落 / 图表；术语在首次出现处批注

MATH Question:

What is the distance, in units, between the points $(2, -6)$ and $(-4, 3)$? Express your answer in simplest radical form.

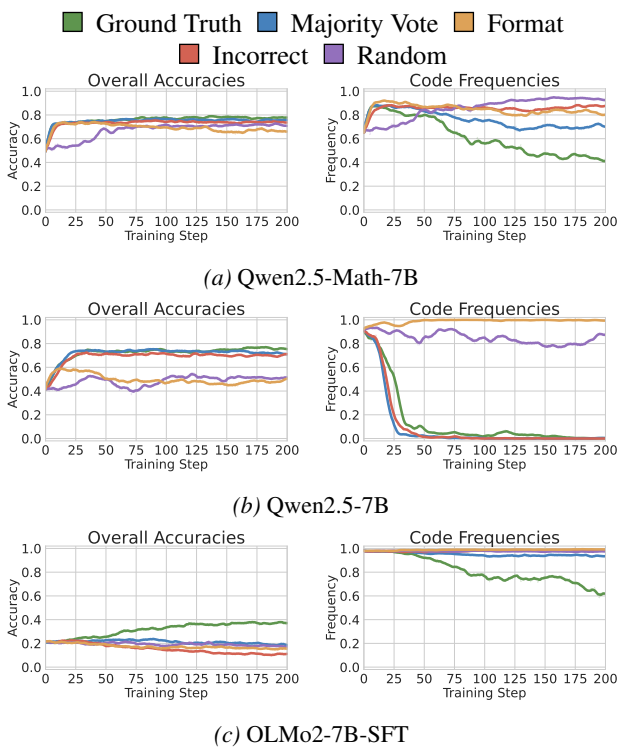
Qwen2.5-Math-7B Solution (correct):

To find the distance between two points (x_1, y_1) and (x_2, y_2) in a Cartesian plane...
Let's break this down step-by-step and compute the result using Python.

```
import math
...
# Calculate the distance using the
    distance formula
distance = math.sqrt(dx**2 + dy**2)
print(distance)
```

output: 10.816653826391969
...
Thus, the final answer is: $\boxed{3\sqrt{13}}$

07.01 *Figure 5.* Example of code reasoning (see Figure 22 for the complete response). Note that both the code and the code execution result are autoregressively generated by Qwen2.5-Math-7B. **No external code interpreter was provided to the model.**



(a) Qwen2.5-Math-7B

(b) Qwen2.5-7B

(c) OLMo2-7B-SFT

07.04 *Figure 6.* We track MATH-500 accuracy (left) and the proportion of answers containing Python code (right) for a Qwen2.5-Math-7B model trained with different reward signals. Under weak (format) or spurious (random, incorrect) rewards, both accuracy and code frequency increase. In contrast, training with ground-truth rewards improves accuracy while code frequency eventually declines, suggesting a different improvement mechanism.

5.2. RLVR with Spurious Rewards Upweight Pre-existing Reasoning Strategies

07.02 Motivated by our observations above, we traced changes in the reasoning behavior of models throughout the course of RLVR training across two dimensions: (1) **Accuracy**: the average accuracy of the model on MATH-500, and (2) **Code reasoning frequency**: the percentage of model responses containing the string "`python`". We find that rewards that we employed in the paper, including spurious rewards such as the random and incorrect rewards, gain much of the accuracy on the Qwen2.5-Math and Qwen2.5 through eliciting the correct reasoning strategy.

07.03 *Table 1.* We report the fraction of MATH-500 responses containing Python code before RL training, along with accuracy for code-based and natural-language-only responses. Qwen2.5-Math models achieve higher accuracy when using code, whereas other models do not benefit from code reasoning. The remaining models (Qwen2.5-1.5B, OLMo2-7B, Llama3.1-8B-Instruct, Llama3.2-3B-Instruct) never generate code.

Model	Qwen2.5-Math-7B	Qwen2.5-Math-1.5B	Qwen2.5-7B	OLMo2-7B-SFT
Code Frequency	65.0	53.6	92.2	98.0
Acc. w/ Code	60.9	52.6	39.9	21.0
Acc. w/ Lang	35.0	17.2	61.5	40.0

07.05 **Performance correlates with code reasoning frequency.** As shown in Figure 6, prior to RLVR training, Qwen2.5-Math-7B uses code reasoning in 65.0% of MATH-500 solutions. Under RLVR with weak or spurious rewards, code frequency rapidly increases (to ∼90% within 15 steps and up to 95.6% for random rewards), closely tracking accuracy gains. With ground-truth rewards, code frequency also rises initially but later declines as natural-language reasoning improves, suggesting learning beyond reliance on code. For *Bad-Code* models, performance gains instead correlate with reduced code usage. Additional analysis of these strategy shifts is provided in Appendix H, indicating that gains from spurious rewards largely arise when models transition from text-based to code-based reasoning.

5.3. Intervening Explicitly on Code Reasoning Frequency

07.06 We have shown observationally that code reasoning frequency increases during RLVR and correlates with improved test performance. Here, we study the effect of explicitly inducing or suppressing code reasoning. We present results for inducing additional code reasoning in the main text,

07.01 图5·示例含义

距离题先写Python，再输出浮点结果，最后化为3√13。所有文本自回归生成，output字样不能视作真实工具验证。

07.02 5.2·测量

同时记录准确率与包含字符串python的回答占比。后者是代码推理的操作性代理指标，不保证代码完整、有效或真的执行。

07.03 表1·条件准确率

Qwen-Math-7B代码/语言回答为60.9%/35.0%；通用Qwen-7B为39.9%/61.5%。不同题可能选择不同策略，组间差异本身不是策略的因果效应。

07.04 图6·关键分叉

弱/虚假奖励下代码频率和准确率同步升；真实奖励后期代码频率下降、准确率继续升，提示真实监督还能带来其他改进机制。

07.05 轨迹·发现

Qwen-Math代码频率由65%快速升至约90%以上；Bad-Code模型的涨分反而常伴随代码减少。RL可能在重分配已有策略的使用概率。

07.06 5.3·从观察到干预

相关曲线之后，作者进一步主动诱导或抑制代码策略，减少仅凭相关性解释机制的局限。抑制实验见附录F。

Table 2. Model performance on MATH-500 after augmenting the prompt to incentivize code reasoning. In this experiment, we force the model's first generated sentence to be "Let's solve this using Python." When applied to Qwen2.5-Math models, which have strong code reasoning priors, our "code-forcing" prompting strategy results in significantly increased test accuracy.

Model	Original	Prompting	Abs. Diff.
Qwen2.5-Math-1.5B	36.2%	60.4%	+24.2%
Qwen2.5-Math-7B	49.4%	64.4%	+15.0%
Qwen2.5-1.5B	3.0%	13.0%	+10.0%
Qwen2.5-7B	41.6%	22.2%	–19.4%
Llama3.2-3B-Instruct	36.8%	8.2%	–28.6%
Llama3.1-8B-Instruct	36.8%	15.2%	–21.6%
OLMo2-7B	9.0%	7.8%	–1.2%
OLMo2-7B-SFT	21.4%	18.6%	–2.8%

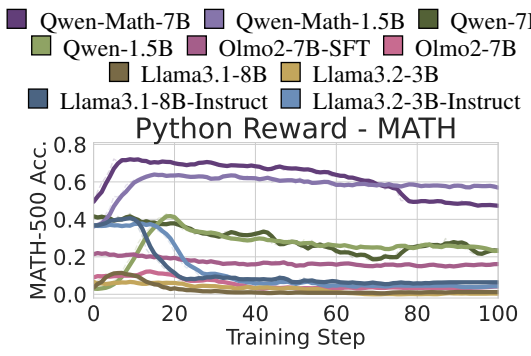


Figure 7. Performance when using our Python reward to explicitly encourage the model to perform code reasoning. This improves performance *only* in Qwen2.5-Math. Qwen2.5-Math-7B starts to generate > 99% code reasoning in 20 training steps.

while experiments suppressing code reasoning are deferred to Appendix F. Across both directions, these interventions produce outcomes consistent with our hypothesis.

Inducing code reasoning improves Qwen2.5-Math models but degrades others. We induce code reasoning via (1) prompting and (2) RLVR. With prompting (forcing responses to begin with "LET'S SOLVE THIS USING PYTHON."), MATH-500 accuracy improves by 24.2%, 15.0%, and 10.0% for Qwen2.5-Math-1.5B, Qwen2.5-Math-7B, and Qwen2.5-1.5B, respectively (Table 2), while performance degrades for other models such as Llama and OLMo2—consistent with their lack of effective code reasoning (Section 5.1).

To induce code reasoning with RLVR, we assign a positive reward if and only if a response contains the string "`python`" (a *Python reward*). This rapidly increases code usage in Qwen2.5-Math-7B to over 99% within 20 training steps (Figure 7) and yields performance gains primarily in the Qwen2.5-Math. For these models, RLVR-based induction matches or exceeds the gains achieved through prompting, while other models show limited improvement.

6. Related Work

Unsupervised Reinforcement Learning. Several approaches have explored unsupervised reinforcement learning. Prasad et al. (2024) introduced Self-Consistency Preference Optimization (ScPO), which trains models to prefer consistent answers over inconsistent ones on unsupervised problems. Similarly, Test-Time Reinforcement Learning (TTRL) (Zuo et al., 2025) leverages majority voting across sampled outputs to estimate pseudo-rewards, demonstrating significant performance improvements on mathematical reasoning tasks. EMPO (Zhang et al., 2025) adapts PPO (Schulman et al., 2017) or GRPO (Shao et al., 2024) for unsupervised RLVR by calculating the rewards based on minimizing the entropy of queries in the semantic space. These approaches suggest that internal model consistency can serve as an effective proxy for correctness, though they do not systematically investigate how different kinds of training rewards affect various model families during RLVR.

Reinforcement Learning for Language Models. The development of language model capabilities has been significantly advanced through reinforcement learning approaches. RLHF has become a standard technique for aligning models with human preferences (Ouyang et al., 2022; Bai et al., 2022), while RLVR has proven effective for tasks with deterministic answers (Guo et al., 2025; Gao et al., 2024; Wen et al., 2025; Lambert et al., 2024; Luo et al., 2025). These methods traditionally rely on accurate supervision signals, either through human feedback or verifiable rewards. Recent work has explored reducing the dependence on human annotations through AI feedback mechanisms (Bai et al., 2022) and through training dynamics analysis (Zhao et al., 2025a), which supports the finding that RL primarily amplifies behaviors or capabilities already buried in the pretrained models (Liu et al., 2025; Yue et al., 2025).

7. Conclusion

Our findings have three main implications: base model pretraining significantly affects RLVR outcomes; even corrupted or spurious supervision can enhance reasoning when it triggers useful existing behaviors; and effects observed in one model family may not generalize to others. Our work highlights the importance of (1) testing across multiple models with differing pretraining distributions, and (2) testing across multiple different baselines, such as format and random rewards, when evaluating reinforcement learning techniques.

This work does not advocate the use of spurious rewards as a training strategy for improving model capabilities. Instead, spurious rewards are introduced solely for controlled analysis, as their misuse in deployed systems could lead to unreliable or undesirable behavior. Overall, the anticipated

08.01 表2·提示干预

固定首句引导写Python，不更新参数。Math-7B提升15.0点，但通用Qwen-7B下降19.4点，表明同一策略诱导可因基模而反向。

08.02 图7·奖励干预

只要含python就得正奖励，Math-7B约20步达到超过99%的代码频率；明显成绩收益主要在Qwen-Math，策略频率本身不是能力指标。

08.03 干预·推论

提示诱导和RL诱导都支持“激发有用先验”的解释；提示改动也会影响回答分布，证据不能证明所有随机奖励收益都由代码单独中介。

08.04 实施·Python奖励

奖励依赖字符串而非数学正确性，也不验证代码执行结果。它是有意针对行为的奖励，与真正独立随机奖励不同。

08.05 相关工作·无标签RL

自一致性、多数投票、语义熵等把模型内部一致性当正确性的代理。本文增加的视角是：必须跨模型并用虚假奖励检验代理的实际贡献。

08.06 相关工作·定位

RLHF依赖偏好，RLVR依赖可验证任务结果；本文研究更弱训练信号，强调预训练行为与优化配置可能共同造成涨分。

08.07 结论·三点

预训练影响RL结果；有损监督也可能触发有用已有行为；单一家族成功不保证迁移。评估新方法应同时比较多模型、多奖励基线。

08.08 结论·用途

虚假奖励用于控制分析，作者不建议把它当能力训练方案。本文最有价值的是提醒：准确率上升不足以单独识别有效监督的来源。

蓝色数字对应段落 / 图表；术语在首次出现处批注

impact of this work is to improve the rigor and robustness of reinforcement learning research rather than to enable new applications.

Acknowledgments

We thank Hamish Ivison and Trung Vu for insightful discussions. This research was developed in part with funding from the Defense Advanced Research Projects Agency's (DARPA) SciFy program (Agreement No. HR00112520300). The views expressed are those of the author and do not reflect the official policy or position of the Department of Defense or the U.S. Government. This work was supported by the Singapore National Research Foundation and the National AI Group in the Singapore Ministry of Digital Development and Information under the AI Visiting Professorship Programme (award number AIVP-2024-001), and by the AI2050 program at Schmidt Sciences. This work was supported by NSF grants 2112471 and 2134012. SG is supported by the NSF GRFP. SSD acknowledges the support of NSF IIS-2110170, NSF DMS-2134106, NSF CCF-2212261, NSF IIS-2143493, NSF CCF-2019844, NSF IIS-2229881, and the Sloan Research Fellowship.

References

AI, E., :, Shah, D. J., Rushton, P., Singla, S., Parmar, M., Smith, K., Vanjani, Y., Vaswani, A., Chaluvaraju, A., Hojel, A., Ma, A., Thomas, A., Polloreno, A., Tanwer, A., Sibai, B. D., Mansingka, D. S., Shivaprasad, D., Shah, I., Stratos, K., Nguyen, K., Callahan, M., Pust, M., Iyer, M., Monk, P., Mazarakis, P., Kapila, R., Srivastava, S., and Romanski, T. Rethinking reflection in pre-training, 2025. URL https://arxiv.org/abs/2504.04022.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., Kerr, J., Mueller, J., Ladish, J., Landau, J., Ndousse, K., Lukosuite, K., Lovitt, L., Sellitto, M., Elhage, N., Schiefer, N., Mercado, N., DasSarma, N., Lasenby, R., Larson, R., Ringer, S., Johnston, S., Kravec, S., Showk, S. E., Fort, S., Lanham, T., Telleen-Lawton, T., Conerly, T., Henighan, T., Hume, T., Bowman, S. R., Hatfield-Dodds, Z., Mann, B., Amodei, D., Joseph, N., McCandlish, S., Brown, T., and Kaplan, J. Constitutional ai: Harmlessness from ai feedback, 2022. URL https://arxiv.org/abs/2212.08073.

Choshen, L., Fox, L., Aizenbud, Z., and Abend, O. On the weaknesses of reinforcement learning for neural machine translation, 2020. URL https://arxiv.org/abs/1907.01752.

Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A. S., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Rozière, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., tian Cantón Ferrer, C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E. A., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I. M., Misra, I., Evtimov, I., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J.-Q., Alwala, K. V., Upasani, K., Plawiak, K., Li, K., neth Heafield, K.-., Stone, K. R., El-Arini, K., Iyer, K., Malik, K., ley Chiu, K., Bhalla, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M. H. M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., lay Bashlykov, N., Bogoychev, N., Chatterji, N. S., Duchenne, O., cCelebi, O., Alrassy, P., Zhang, P., Li, P., Vasić, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Girdhar, R., Patel, R., main Sauvestre, R., nie Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., hana Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S. C., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., ginie Do, V., Vogeti, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., ney Meers, W., Martinet, X., Wang, X., Tan, X. E., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A. K., Grattafiori, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Vaughan, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Franco,

A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, P.-Y. B., Loyd, B., de Paola, B., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Civin, D., Beaty, D., Kreymer, D., Li, S.-W., Wyatt, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Ozgenel, F., Caggioni, F., Guzm'an, F., Kanayet, F. J., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Thattai, G., Herman, G., Sizov, G. G., Zhang, G., Lakshminarayanan, G., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Molybog, I., Tufanov, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., KamHou, U., Saxena, K., Prasad, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Huang, K., Chawla, K., Lakhotia, K., Huang, K., Chen, L., Garg, L., Lavender, A., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Tsimpoukelli, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Laptev, N. P., Dong, N., Zhang, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., dro Rittner, P., Bontrager, P., Roux, P., Dollár, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Li, R., Hogan, R., Battey, R., Wang, R., Maheswari, R., Howes, R., Rinott, R., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Shankar, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Cho, S.-B., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Kohler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V. A., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wang, X., Wu, X., Wang, X., Xia, X., Wu, X., Gao, X., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Wang, Y., Hao, Y., Qian, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., and Zhao, Z. The llama 3 herd of models. *ArXiv*, abs/2407.21783, 2024. URL https://api.semanticscholar.org/CorpusID:271571434.

Gandhi, K., Chakravarthy, A., Singh, A., Lile, N., and Goodman, N. D. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars. *arXiv preprint arXiv:2503.01307*, 2025.

Gao, J., Xu, S., Ye, W., Liu, W., He, C., Fu, W., Mei, Z., Wang, G., and Wu, Y. On designing effective rl reward at training time for llm reasoning. *arXiv preprint arXiv:2410.15115*, 2024.

Gao, Z., Chen, L., Zhou, J., and Dai, B. One-shot entropy minimization, 2025. URL https://arxiv.org/abs/2505.20282.

Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Ding, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Chen, J., Yuan, J., Tu, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., You, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Zhou, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. Deepseek-r1 incentivizes

reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Hochlehnert, A., Bhatnagar, H., Udandarao, V., Albanie, S., Prabhu, A., and Bethge, M. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility, 2025. URL https://arxiv.org/abs/2504.07086.

Hu, J., Wu, X., Shen, W., Liu, J. K., Zhu, Z., Wang, W., Jiang, S., Wang, H., Chen, H., Chen, B., Fang, W., Xianyu, Cao, Y., Xu, H., and Liu, Y. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework, 2025a. URL https://arxiv.org/abs/2405.11143.

Hu, J., Zhang, Y., Han, Q., Jiang, D., Zhang, X., and Shum, H.-Y. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025b.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, S., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Bras, R. L., Tafjord, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. Tülu 3: Pushing frontiers in open language model post-training. 2024.

Li, J., Beeching, E., Tunstall, L., Lipkin, B., Soletskyi, R., Huang, S. C., Rasul, K., Yu, L., Jiang, A., Shen, Z., Qin, Z., Dong, B., Zhou, L., Fleureau, Y., Lample, G., and Polu, S. Numinamath. [https://huggingface.co/AI-MO/NuminaMath-CoT](https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf), 2024.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2023.

Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

Luo, M., Tan, S., Wong, J., Shi, X., Tang, W. Y., Roongta, M., Cai, C., Luo, J., Li, L. E., Popa, R. A., and Stoica, I. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a 2025. Notion Blog.

OLMo, T., Walsh, P., Soldaini, L., Groeneveld, D., Lo, K., Arora, S., Bhagia, A., Gu, Y., Huang, S., Jordan, M., Lambert, N., Schwenk, D., Tafjord, O., Anderson, T., Atkinson, D., Brahman, F., Clark, C., Dasigi, P., Dziri, N., Guerquin, M., Ivison, H., Koh, P. W., Liu, J., Malik, S., Merrill, W., Miranda, L. J. V., Morrison, J. D., Murray, T. C., Nam, C., Pyatkin, V., Rangapur, A., Schmitz, M., Skjonsberg, S., Wadden, D., Wilhelm, C., Wilson, M., Zettlemoyer, L. S., Farhadi, A., Smith, N. A., and Hajishirzi, H. 2 olmo 2 furious. *ArXiv*, abs/2501.00656, 2024. URL https://api.semanticscholar.org/CorpusID:275213098.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

Prabhudesai, M., Chen, L., Ippoliti, A., Fragkiadaki, K., Liu, H., and Pathak, D. Maximizing confidence alone improves reasoning, 2025. URL https://arxiv.org/abs/2505.22660.

Prasad, A., Yuan, W., Pang, R. Y., Xu, J., Fazel-Zarandi, M., Bansal, M., Sukhbaatar, S., Weston, J., and Yu, J. Self-consistency preference optimization. *arXiv preprint arXiv:2411.04109*, 2024.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting, 2024. URL https://arxiv.org/abs/2310.11324.

Shafayat, S., Tajwar, F., Salakhutdinov, R., Schneider, J., and Zanette, A. Can large reasoning models self-train?, 2025. URL https://arxiv.org/abs/2505.21444.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo,

D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Wang, J., Fang, M., Wan, Z., Wen, M., Zhu, J., Liu, A., Gong, Z., Song, Y., Chen, L., Ni, L. M., Yang, L., Wen, Y., and Zhang, W. Openr: An open source framework for advanced reasoning with large language models, 2024. URL https://arxiv.org/abs/2410.09671.

Wang, S., Yu, L., Gao, C., Zheng, C., Liu, S., Lu, R., Dang, K., Chen, X., Yang, J., Zhang, Z., Liu, Y., Yang, A., Zhao, A., Yue, Y., Song, S., Yu, B., Huang, G., and Lin, J. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning, 2025a. URL https://arxiv.org/abs/2506.01939.

Wang, Y., Yang, Q., Zeng, Z., Ren, L., Liu, L., Peng, B., Cheng, H., He, X., Wang, K., Gao, J., Chen, W., Wang, S., Du, S. S., and Shen, Y. Reinforcement learning for reasoning in large language models with one training example, 2025b. URL https://arxiv.org/abs/2504.20571.

Wen, L., Cai, Y., Xiao, F., He, X., An, Q., Duan, Z., Du, Y., Liu, J., Tang, L., Lv, X., Zou, H., Deng, Y., Jia, S., and Zhang, X. Light-r1: Curriculum SFT, DPO and RL for long COT from scratch and beyond. In Rehm, G. and Li, Y. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp. 318–327, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-288-6. doi: 10.18653/v1/2025.acl-industry.24. URL https://aclanthology.org/2025.acl-industry.24/.

Xie, T., Gao, Z., Ren, Q., Luo, H., Hong, Y., Dai, B., Zhou, J., Qiu, K., Wu, Z., and Luo, C. Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768*, 2025.

Yang, A., Zhang, B., Hui, B., Gao, B., Yu, B., Li, C., Liu, D., Tu, J., Zhou, J., Lin, J., Lu, K., Xue, M., Lin, R., Liu, T., Ren, X., and Zhang, Z. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024a. URL https://arxiv.org/abs/2409.12122.

Yang, Q. A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Dong, G., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y.-C., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., Qiu, Z., Quan, S., and Wang, Z. Qwen2.5 technical report. *ArXiv*, abs/2412.15115, 2024b. URL https://api.semanticscholar.org/CorpusID:274859421.

Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., YuYue, Dai, W., Fan, T., Liu, G., Liu, J., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, R., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W.-Y., Zhang, Y.-Q., Yan, L., Wu, Y., and Wang, M. DAPO: An open-source LLM reinforcement learning system at scale. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=2a36EMSSTp.

Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Song, S., and Huang, G. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

Zeng, W., Huang, Y., Liu, Q., Liu, W., He, K., Ma, Z., and He, J. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.

Zhang, Q., Wu, H., Zhang, C., Zhao, P., and Bian, Y. Right question is already half the answer: Fully unsupervised llm reasoning incentivization. *arXiv preprint arXiv:2504.05812*, 2025.

Zhao, R., Meterez, A., Kakade, S., Pehlevan, C., Jelassi, S., and Malach, E. Echo chamber: Rl post-training amplifies behaviors learned in pretraining. *arXiv preprint arXiv:2504.07912*, 2025a.

Zhao, X., Kang, Z., Feng, A., Levine, S., and Song, D. Learning to reason without external rewards. *arXiv preprint arXiv:2505.19590*, 2025b.

Zuo, Y., Zhang, K., Qu, S., Sheng, L., Zhu, X., Qi, B., Sun, Y., Cui, G., Ding, N., and Zhou, B. Ttrl: Test-time reinforcement learning. *arXiv preprint arXiv:2504.16084*, 2025.

Appendix

A Experimental Setup **14**

A.1 Preliminaries on GRPO . . . 14
A.2 Datasets and Models . . . 14
A.3 Training Configuration . . . 14
A.4 Decoding Configuration . . . 14
A.5 Computation Resource . . . 14
A.6 Visualization . . . 15
A.7 Experimental Setups for TTRL and One-Shot RL . . . 15

B The Curious Case of Random Rewards **15**

B.1 Conjecture: Clipping Bias Brings Training Signals under Random Rewards . . . 16
B.2 High Stochasticity in RLVR Without Clipping . . . 20

C Additional Results on AMC **21**

D Additional Results on AIME 2024 and AIME 2025 **21**

D.1 Spurious Rewards Yield Significant RLVR Gains on Qwen2.5-Math . . . 22
D.2 (Lack of) Generalization to Other Models . . . 22

E Examination of Existing Methods on Models Beyond Qwen **22**

F Compound Rewards that Inhibit Code Reasoning **22**

G Beyond Code Reasoning: Another Beneficial Pattern that RLVR Can Easily Elicit **23**

H Switch of Reasoning Strategies during RLVR **24**

I Spurious Prompts: Qwen2.5-Math-7B's Unreasonably High Sensitivity to Prompts **25**

I.1 Existing Prompts . . . 25
I.2 Spurious Prompts . . . 26

J Additional Results on Models That Have Undergone RL Training **26**

K Qualitative Analysis on Qwen2.5-Math-7B's Coding Reasoning Behaviors **26**

A. Experimental Setup

A.1. Preliminaries on GRPO

Denote the input prompt as x and the corresponding model rollouts as y. Denote the t-th token in the i-th rollout y as y_t, where $1 \le t \le |y|$. GRPO loss, as introduced by (Shao et al., 2024), contains the following components: For each prompt x, we compute the group-wise mean $\bar{r}_x$ and standard deviation σ_x. The normalized *group-relative advantage* is $\hat{A}(x,y) = \frac{r(x,y)-\bar{r}_x}{\sigma_x}$. Let π_{old} be the behavior policy that produced the trajectories and π_{ref} be a frozen reference policy (for example, the initial supervised model). We denote the token-level importance ratio by $\rho_t(y;\theta) = \frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\text{old}}(y_t|x,y_{<t})}$, where t is the token index. With PPO-style clipping threshold ϵ_c, KL-penalty weight λ, the surrogate objective maximized is

$$
\begin{aligned}
J(\theta) =& \mathbb{E}_{x\sim\mathcal{D},y\sim\pi_{\text{old}}(\cdot|x)}\left[\sum_{t=1}^{|y|}\min\Big(\rho_t(y;\theta)\,\hat{A}(x,y),\,\text{clip}\big(\rho_t(y;\theta),1-\epsilon_c,1+\epsilon_c\big)\,\hat{A}(x,y)\Big)\right] \\
&- \lambda\,\mathbb{E}_{x\sim\mathcal{D}}\Big[\text{KL}\big(\pi_\theta\|\pi_{\text{ref}}\big)\Big].
\end{aligned} \tag{3}
$$

The KL regularization is typically adopted to ensure our model does not deviate too far from the frozen reference model. However, since we focus primarily on verifiable rewards and investigate the signals produced by these rewards, we assume that distribution shift is of lesser concern following (Liu et al., 2025). Moreover, KL regularization adds confounding factors to our analysis. Recent work has also shown that removing the KL term leads to better performance (Hu et al., 2025b). Therefore, we set $\lambda = 0$ in our work.

A.2. Datasets and Models

We conduct our experiments on three canonical mathematical reasoning benchmarks:

- **MATH-500** (Hendrycks et al., 2021; Lightman et al., 2023): A standardized subset of the MATH dataset focusing on advanced mathematical reasoning, including problems from algebra, calculus, and number theory.
- **AMC** (Li et al., 2024): The American Mathematics Competition dataset contains diverse mathematical problems ranging from algebra to combinatorics and geometry. For this benchmark, we report model's average performance over 8 trials (avg@8).
- **AIME** (Li et al., 2024): The American Invitational Mathematics Examination dataset consists of challenging high-school level mathematical problems requiring multi-step reasoning. We include AIME questions from 2024 and 2025 for evaluation. For this benchmark, we report model's average performance over 8 trials (avg@8).

For our initial experiments, we primarily utilize the Qwen2.5-Math-7B model, a 7 billion parameter language model specifically tuned for mathematical reasoning tasks. We select this model (1) for its strong baseline performance on mathematical problems while remaining computationally efficient for multiple experimental iterations and (2) because it is frequently used in prior work (Zuo et al., 2025; Wang et al., 2025b).

A.3. Training Configuration

Unless otherwise specified, we train each model on 8 GPUs with a constant learning rate of 5e-7, a mini batch size (number of rollouts seen before a gradient update) of 128, and a rollout batch size (number of prompts we rollout at the same time) of 64. For each prompt, we collect 16 rollouts to compute advantages for GRPO update. We use a sampling temperature $\tau = 1$. We do not apply KL divergence loss or entropy loss in our training.

A.4. Decoding Configuration

We use temperature of 0.0 for pass at 1, and temperature of 0.6 for pass at k during decoding time. All other hyperparameters are the default value following the Zuo et al. (2025) codebase.

A.5. Computation Resource

Each RLVR run in our experiment takes approximately 24 hours on 8 A100s.

14.01 A.1·GRPO完整定义

每题采样一组回答，以组均值中心化、组标准差归一化，得到回答级优势；再用同一优势更新该回答的token。

14.02 策略区分

π_old是本轮生成轨迹的行为策略，π_ref是冻结参考模型；前者用于重要性比，后者用于KL约束，两者作用不同。

14.03 目标式3

KL通常约束当前模型偏离冻结参考模型；本篇为减少分析混杂令λ=0。因此这里的训练不能默认有参考模型拉回作用。

14.04 A.2·评测集合

MATH-500、AMC和AIME难度与规模不同。AMC/AIME均报告8次试验的平均准确率，不能解释成8次挑最好答案。

14.05 模型选择

初始用7B数学模型是为了兼顾性能、成本及与既有工作的可比性；随后跨家族检验，避免选择本身决定结论。

14.06 A.3·关键超参

8 GPU、学习率5×10^-7；每轮64题×16回答=1024条，minibatch为128条回答，因而一轮通常含8次小批更新；不加KL和熵损失。

14.07 A.4·解码

pass@1温度0，采样评测温度0.6，训练采样温度1。正文明确AMC/AIME为average@8，附录笼统的pass at k表述不应改变指标定义。

14.08 A.5·计算量

每次运行约8张A100训练24小时。跨方法比较需同时看更新次数与总计算，而不仅看横轴训练步。

蓝色数字对应段落 / 图表；术语在首次出现处批注

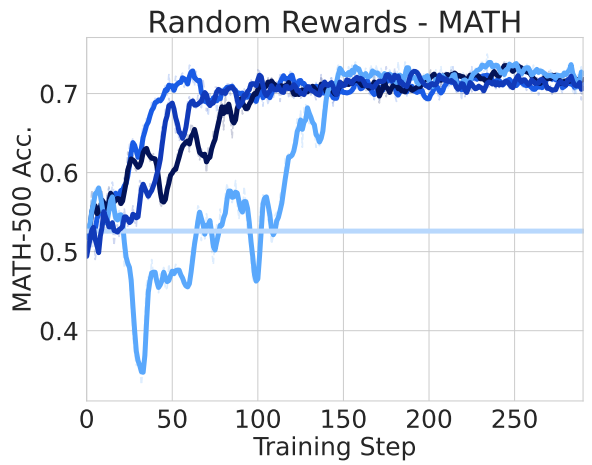


Figure 8. We train Qwen2.5-Math-7B using GRPO with random rewards of different probability $\gamma \in \{0.7, 0.5, 0.3, 0.001, 0\}$. We note that with the exception of $\gamma = 0$, which leads the model to not perform any learning, all other probability configuration successfully leads the model to achieve a significant performance gain after some period of random walk. Furthermore, once in the high-accuracy regime, the training procedure maintain model's performance.

- Random $\gamma = 0.7$
- Random $\gamma = 0.5$
- Random $\gamma = 0.3$
- Random $\gamma = 0.001$
- Random $\gamma = 0$

A.6. Visualization

In all plots, we smooth the metric of interest across ten training steps, and overlay the raw curves as transparent dashed lines. We do not smooth the step 0 performance, which corresponds to the performance of the base model before training. We report the smoothed value at the final training step as the final performance.

A.7. Experimental Setups for TTRL and One-Shot RL

TTRL setup. We follow all hyperparameters from the original TTRL paper (Zuo et al., 2025), and train all models using their publicly released implementation. Due to compute constraints, we do not extensively sweep hyperparameters on the new base models we consider. Note that the TTRL accuracies are not directly comparable to our majority-vote reward results, because (a) TTRL trains on (unlabeled) test prompts while we train on a much larger set of distinct train prompts, and (b) TTRL updates labels during training based on the online policy's majority vote, while we assign labels once offline.

Differences in our One-Shot RL setup from (Wang et al., 2025b). We note that for the one-shot RL settings in our paper, we do not apply entropy loss for more consistent setups with other experiments and better stability. However, (Wang et al., 2025b) shows that the entropy loss may further improve the performance of one-shot RL and post-saturation generalization. We use the same training example π_1 (defined in (Wang et al., 2025b)) for one-shot RLVR experiments on all models as in (Wang et al., 2025b). Their work discusses that different models should possibly select different examples for more performant one-shot RLVR training, but use π_1 in all experiments due to high cost of trying different possible 1-shot examples. In particular, we note that this setup difference may cause a noticeable performance difference for Qwen2.5-7B model compared to the entropy loss setting used in (Wang et al., 2025b) (71.2% with entropy loss vs. 54.8% without).

B. The Curious Case of Random Rewards

In this section, we study a special case of our spurious rewards, random rewards, where the reward is randomly assigned independently of the model rollouts. We first show that random rewards work across several non-zero reward probabilities (Figure 8), confirming that this is a stable observation. Next, we provide detailed gradient derivation in this special case and discuss our hypothesis on one of the potential sources of training signals with random rewards—the clipping bias in GRPO update.

Random rewards with varying probabilities consistently improve performance. We train Qwen2.5-Math-7B using GRPO with random rewards assigned by $\texttt{Bernoulli}(\gamma)$ variables, where $\gamma \in \{0.7, 0.5, 0.3, 0.001, 0\}$. Each response receives reward 1 with probability γ and 0 otherwise. We find that all non-zero probability configurations successfully lead to significant performance gains after an initial period of exploration, yielding comparable performance to ground truth rewards (Figure 8). $\gamma = 0$ yields no improvement as expected, since constant rewards provide no learning signal. The convergence speed varies with γ, but all configurations eventually reach similar high-performance regimes, with accuracy improvements of 15-20 percentage points on MATH-500.

15.01 图8·随机概率消融

γ取0.001、0.3、0.5、0.7均有提升但速度不同；γ=0不更新。此处“非零概率”只针对已测试值，γ=1同样是常数奖励。

15.02 A.6·平滑口径

曲线用10步平滑，初始step0不平滑，最终结果取最后一步的平滑值。不要把某个原始尖峰当作报告成绩。

15.03 A.7·TTRL可比性

TTRL用未标注测试题并在线更新多数标签；本文多数票用独立训练题、离线固定标签。因此两类成绩不能直接归因于标签机制优劣。

15.04 单样本RL·复现差异

本篇去除原方法熵损失，并对所有模型用同一个训练样例、未充分调参；这些差异可能影响跨模型复测结果。

15.05 附录B·路线

先检验随机概率，再推导裁剪偏置。奖励独立于输出是这个特殊理论分析的关键前提。

15.06 现象·稳健范围

不同γ在随机探索一段时间后可进入高准确率区域；这是本模型与配置下的经验规律，不是随机奖励普遍有效的保证。

16.01 **Recap of notations.** Recall that we denote the input prompt as x and the corresponding model rollouts as $\{y^{(1)}, \cdots, y^{(G)}\}$, where $G \in \mathbb{N}^+$ is the rollout size. In addition, we denote the t-th token in the i-th rollout $y^{(i)}$ as $y_t^{(i)}$, where $t \in \mathbb{N}^+$ and $1 \le t \le |y^{(i)}|$. The normalized group-relative advantage in GRPO is $\hat{A}(x, y) = \frac{r(x,y)-\bar{r}_x}{\sigma_x}$, where $\bar{r}_x$ is the group-wise mean and σ_x is the group-wise standard deviation. The token-level importance ratio[4] is $\rho_t(y;\theta) = \frac{\pi_{\theta,x}(y_t)}{\pi_{\text{old},x}(y_t)}$, where t is the token index.

B.1. Conjecture: Clipping Bias Brings Training Signals under Random Rewards

B.1.1. Expected Advantage of Random Rewards

16.02 We investigate the expected advantage when the reward signal is assigned by a random $\texttt{Bernoulli}(\gamma)$ variable. For a prompt x and rollouts $\{y^{(i)}\}_{i=1}^G$, the reward of each $y^{(i)}$ is $r(x, y^{(i)}) \sim \texttt{Bernoulli}(\gamma)$, so the expected average reward of x is γ. Assuming $\hat{A}_i = 0$ when $\sigma_x = 0$, the sum of the normalized advantages over G rollouts $\sum_{i=1}^G \hat{A}(x, y^{(i)}) := \sum_{i=1}^G \frac{r(x,y^{(i)})-\bar{r}_x}{\sigma_x} = \frac{\sum_{i=1}^G r(x,y^{(i)})-G\cdot\bar{r}_x}{\sigma_x} = 0$ by construction. Furthermore, for i.i.d. rewards that are independent of the provided samples, such as the random rewards used in our experiments, $\mathbb{E}(\sum_{i=1}^G \hat{A}(x, y^{(i)})) = G \cdot \mathbb{E}(\hat{A}) = 0 \implies \mathbb{E}(\hat{A}) = 0$.

16.03 The clipping term in GRPO loss (Equation 3) prevents excessive deviation from the previous policy, stabilizing training. Recent work suggests this term, which operates on the ratio $\rho_t(y;\theta) = \pi_{\theta,x}(y_t)/\pi_{\text{old},x}(y_t)$, introduces bias toward exploitation in the case of ground truth reward (Yu et al., 2025). Here, we analyze the bias in the settings of the random rewards, where each rollout receives an advantage that is independent of the rollout.

16.04 We note that the loss is no longer differentiable everywhere with clipping. In practical implementations, e.g., PyTorch, the gradients will be automatically set to 0 when the value is clipped. For simplicity, we discuss the gradient of the loss function by assuming 0 gradient at the non-differentiable points.

B.1.2. Gradient Derivation of Clipping Bias under Random Rewards

16.05 In this section, we derive the training signals induced by the clipping factor, which we name as a "clipping bias". Specifically, we define the clipping bias as the difference in the expected gradients after adding clipping to the GRPO objective[5]:

$$\texttt{Bias}(\nabla_\theta J(\theta)) = \mathbb{E}_{x,y}[\nabla_\theta J(\theta)] - \mathbb{E}_{x,y}[\nabla_\theta L^{\text{unclipped}}(\theta)].$$

16.06 As we derived above, $\mathbb{E}(\hat{A}(x, y_j)) = 0$. Assuming a simple loss with no clipping, the expectation of the policy gradient without the clipping term is also trivially zero given that $\hat{A}$ is independent of other variables:

$$\begin{aligned}
&\mathbb{E}_{x,y}[\nabla_\theta L^{\text{unclipped}}(\theta)] \\
=&\mathbb{E}_{x,y}\left[\frac{1}{|y|}\sum_{t=1}^{|y|}\nabla_\theta \rho_t(y;\theta)\,\hat{A}(x,y)\right] \\
=&\mathbb{E}(\hat{A})\mathbb{E}_{x,y}\left[\frac{1}{|y|}\sum_{t=1}^{|y|}\nabla_\theta \rho_t(y;\theta)\right] \\
=&0.
\end{aligned}$$

16.07 Therefore, the clipping bias under random reward is

$$\texttt{Bias}(\nabla_\theta J(\theta)) = \mathbb{E}[\nabla_\theta J(\theta)] - \mathbb{E}[\nabla_\theta L^{\text{unclipped}}(\theta)] = \mathbb{E}[\nabla_\theta J(\theta)].$$

Next, we compute the exact form of this clipping bias, i.e., the gradient in GRPO with random rewards. Recall from Eq. 3, the surrogate objective that we aim to maximize is:

$$J(\theta) = \min\Big(\rho_t(y;\theta)\,\hat{A}(x,y), \text{clip}\big(\rho_t(y;\theta), 1-\epsilon_c, 1+\epsilon_c\big)\,\hat{A}(x,y)\Big).$$

[4]For simplicity, we denote $\pi(y_t|x, y_{<t})$ as $\pi_x(y_t)$, and similarly $\pi_\theta(y_t|x, y_{<t})$ as $\pi_{\theta,x}(y_t)$, $\pi_{\text{old}}(y_t|x, y_{<t})$ as $\pi_{\text{old},x}(y_t)$.
[5]For simplicity, we denote $\mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_{\text{old}}(\cdot|x)}$ as $\mathbb{E}_{x,y}$

16.01 记号回顾

G为每题回答数；优势=(奖励-组均值)/组标准差；ρ是给定题目与前缀的token条件概率比。不要把它看成整条回答概率比。

16.02 B.1.1·零均值

同组优势之和由中心化构造为0；独立同分布且与回答无关的奖励，再由对称性得到每个回答的优势期望为0。全同奖励时约定优势为0。

16.03 裁剪作用

常规目的在于限制策略偏移；此处分析它在随机优势下如何打破正负梯度抵消，而不是在奖励中找正确答案信息。

16.04 数学约定

在不可微的裁剪边界取0梯度，其余按分段函数求导。实现的精确边界次梯度约定需与框架核对，主要结论来自区间内部。

16.05 B.1.2·偏置定义

偏置定义为"有裁剪的期望梯度"减"无裁剪的期望梯度"，不是相对于真实任务最优梯度的统计偏差。

16.06 无裁剪推导

利用优势独立、E[A]=0，拆开期望后梯度为0。当前策略经同一批随机奖励多次更新后也会依赖奖励，故这一独立性是分析的理想化前提。

16.07 推导承接

无裁剪基准为0，因此偏置等于有裁剪目标的期望梯度；下一页按优势正负拆分min函数。零期望不等于样本梯度恒为0。

17.01 For simplicity, we further denote $R_\theta = \rho_t(y;\theta) = \frac{\pi_{\theta,x}(y_t)}{\pi_{\text{old},x}(y_t)}$ to be the token-level importance ratio, $C = \text{clip}(R_\theta, 1-\epsilon_c, 1+\epsilon_c)$ to be the clipping term, and $\hat{A}(x,y)$ as $\hat{A}$. So we have $J(\theta) = \min\big(R_\theta \cdot \hat{A}, C \cdot \hat{A}\big)$. We now analyze the gradient of GRPO loss with respect to the policy model parameters θ based on the sign of $\hat{A}$.

As discussed in Section B.1, the loss function is not differentiable everywhere. Therefore, we discuss the gradients with respect to the differentiable regions below and set the gradients to 0 at the non-differentiable points at the end.

17.02 **Case 1: $\hat{A} \geq 0$.** When $\hat{A} \geq 0$, we can directly take $\hat{A}$ out of the $\min$ function. Thus,

$$J(\theta) = \min\big(R_\theta \cdot \hat{A}, C \cdot \hat{A}\big) = \hat{A} \cdot \min(R_\theta, C).$$

Since C is defined as $C = \text{clip}(R_\theta, 1-\epsilon_c, 1+\epsilon_c)$, we have:

- If $R_\theta < 1-\epsilon_c$, then $C = 1-\epsilon_c$. Thus, $\min(R_\theta, C) = R_\theta$.
- If $1-\epsilon_c \leq R_\theta \leq 1+\epsilon_c$, then $C = R_\theta$. Thus, $\min(R_\theta, C) = R_\theta$.
- If $R_\theta > 1+\epsilon_c$, then $C = 1+\epsilon_c$. Thus, $\min(R_\theta, C) = 1+\epsilon_c$.

Combining these, when $\hat{A} \geq 0$, the objective is:

$$L^+(\theta) = \hat{A} \cdot \begin{cases} R_\theta, & \text{if } R_\theta < 1+\epsilon_c, \\ 1+\epsilon_c, & \text{if } R_\theta > 1+\epsilon_c. \end{cases}$$

The gradient with respect to θ for this case is:

$$\nabla_\theta L^+(\theta) = \hat{A} \cdot \begin{cases} \nabla_\theta R_\theta, & \text{if } R_\theta < 1+\epsilon_c, \\ 0, & \text{if } R_\theta > 1+\epsilon_c. \end{cases}$$

17.03 **Case 2: $\hat{A} < 0$.** When $\hat{A} < 0$, multiplying by $\hat{A}$ flips the $\min$ function. So,

$$L^-(\theta) = \min(R_\theta \cdot \hat{A}, C \cdot \hat{A}) = \hat{A} \cdot \max(R_\theta, C).$$

Applying a similar analysis, when $\hat{A} < 0$, the gradient with respect to θ for this case is:

$$\nabla_\theta L^-(\theta) = \hat{A} \cdot \begin{cases} 0, & \text{if } R_\theta < 1-\epsilon_c, \\ \nabla_\theta R_\theta, & \text{if } R_\theta > 1-\epsilon_c. \end{cases}$$

17.04 **Combining the cases.** Together, the gradient is:

$$\nabla_\theta J(\theta) = \hat{A} \cdot \begin{cases} \nabla_\theta R_\theta, & \text{if } \hat{A} \geq 0 \text{ and } R_\theta < 1+\epsilon_c, \\ 0, & \text{if } \hat{A} \geq 0 \text{ and } R_\theta > 1+\epsilon_c, \\ 0, & \text{if } \hat{A} < 0 \text{ and } R_\theta < 1-\epsilon_c, \\ \nabla_\theta R_\theta, & \text{if } \hat{A} < 0 \text{ and } R_\theta > 1-\epsilon_c. \end{cases}$$

17.01 简写

令R=π_θ/π_old、C=clip(R)、A为优势，目标为min(RA,CA)。推导针对最大化目标，若实现最小化负损失需反转符号。

17.02 情况1·正优势

A≥0时J=A·min(R,C)。R未超过上界时梯度为A□R；超过1+ε后目标封顶，继续提高概率不再获益。

17.03 情况2·负优势

A<0会翻转min关系，J=A·max(R,C)。R低于下界时梯度为0；高于1-ε时保留负优势对应的梯度。

17.04 合并·核心不对称

区间内正负优势都能更新；上界外只留下负优势，下界外只留下正优势。这就是随机正负信号可能无法抵消的原因。

18.01 Then, the clipping bias, $\texttt{Bias}(\nabla_\theta J(\theta))$, is

$$
\begin{aligned}
&\texttt{Bias}(\nabla_\theta J(\theta)) \\
=&\mathbb{E}_{\hat{A},x,y}\left[\nabla_\theta J(\theta)\right] \\
=&P(\hat{A}\geq 0)\cdot\mathbb{E}_{\hat{A}>0,x,y}[\nabla_\theta L^{+}(\theta)] + P(\hat{A}<0)\cdot\mathbb{E}_{\hat{A}<0,x,y}[\nabla_\theta L^{-}(\theta)] + 0 \\
=&\mathbb{E}_{x,y}\left[\begin{cases} P(\hat{A}\geq 0)\cdot\mathbb{E}_{\hat{A}\geq 0}[\hat{A}]\cdot\nabla_\theta R_\theta, & \text{if } R_\theta < 1+\epsilon_c, \\ 0, & \text{if } R_\theta > 1+\epsilon_c. \end{cases}\right] \\
&+\mathbb{E}_{x,y}\left[\begin{cases} 0, & \text{if } R_\theta < 1-\epsilon_c, \\ P(\hat{A}<0)\cdot\mathbb{E}_{\hat{A}<0}[\hat{A}]\cdot\nabla_\theta R_\theta, & \text{if } R_\theta > 1-\epsilon_c. \end{cases}\right] \\
=&\mathbb{E}_{x,y}\left[\begin{cases} P(\hat{A}\geq 0)\cdot\mathbb{E}_{\hat{A}\geq 0}[\hat{A}]\cdot\nabla_\theta R_\theta, & \text{if } R_\theta < 1-\epsilon_c, \\ (P(\hat{A}\geq 0)\cdot\mathbb{E}_{\hat{A}\geq 0}[\hat{A}] + P(\hat{A}<0)\cdot\mathbb{E}_{\hat{A}<0}[\hat{A}])\cdot\nabla_\theta R_\theta, & \text{if } 1-\epsilon_c < R_\theta < 1+\epsilon_c, \\ P(\hat{A}<0)\cdot\mathbb{E}_{\hat{A}<0}[\hat{A}]\cdot\nabla_\theta R_\theta, & \text{if } R_\theta > 1+\epsilon_c. \end{cases}\right]
\end{aligned}
$$

18.02 By definition, $\hat{A}$ is a normalized distribution and $\mathbb{E}(\hat{A})=0$ from Appendix B.1.1, so

$$
\mathbb{E}(\hat{A}) = P(\hat{A}\geq 0)\,\mathbb{E}_{\hat{A}\geq 0}(\hat{A}) + P(\hat{A}<0)\,\mathbb{E}_{\hat{A}<0}(\hat{A}) = 0, \text{ and}
$$

$$
P(\hat{A}\geq 0)\,\mathbb{E}_{\hat{A}\geq 0}(\hat{A}) = -P(\hat{A}<0)\,\mathbb{E}_{\hat{A}<0}(\hat{A}) \geq 0.
$$

18.03 We denote $\mu = P(\hat{A}\geq 0)\,\mathbb{E}_{\hat{A}\geq 0}(\hat{A}) = -P(\hat{A}<0)\,\mathbb{E}_{\hat{A}<0}(\hat{A})$, which by definition is positive. And we substitute $R_\theta = \frac{\pi_{\theta,x}(y_t)}{\pi_{\text{old},x}(y_t)}$ in the conditions. Finally, we set the gradients on non-differentiable points to 0 and have

$$
\texttt{Bias}(\nabla_\theta J(\theta)) = \mu\cdot\mathbb{E}_{x,y}\left[\begin{cases} \nabla_\theta R_\theta, & \text{if } \pi_{\theta,x}(y_t) < \pi_{\text{old},x}(y_t)\cdot(1-\epsilon_c), \\ 0, & \text{if } \pi_{\text{old},x}(y_t)\cdot(1-\epsilon_c) \leq \pi_{\theta,x}(y_t) \\ & \quad \leq \pi_{\text{old},x}(y_t)\cdot(1+\epsilon_c), \\ -\nabla_\theta R_\theta, & \text{if } \pi_{\theta,x}(y_t) > \pi_{\text{old},x}(y_t)\cdot(1+\epsilon_c). \end{cases}\right]
$$

18.04 Recall that $R_\theta = \rho_t(y;\theta) = \frac{\pi_{\theta,x}(y_t)}{\pi_{\text{old},x}(y_t)}$ and both $\pi_{\theta,x}(y_t)$ and $\pi_{\text{old},x}(y_t)$ in range $[0,1]$. Therefore, we observe that there is a *positive gradient* $\big($gradient that increases $\pi_{\theta,x}(y_t)\big)$ if $R_\theta < 1-\epsilon_c$, and a *negative gradient* $\big($gradient that decreases $\pi_{\theta,x}(y_t)\big)$ if $R_\theta > 1+\epsilon_c$, which means the clipping bias discourages the model from leaving the clipping region. In the rest of the paper, for simplicity we refer to the positive gradient case as "positive gradient bias" and the other way as "negative gradient bias".

B.1.3. Clipping Creates Asymmetric Updates Towards Model Prior Knowledge

18.05 In this section, we provide further evidence for our hypothesis that the clipping bias increases the likelihood of high-probability rollouts, similar to Yu et al. (2025)'s analysis on GRPO with ground-truth labels. We showcase this trend with a simple example. Consider a case where a token has a high-probability $\pi_{\text{old},x}(y_t) = 0.85$ and an $\epsilon_c = 0.2$ that we adopt in our experiments. Then, the upper threshold from the bias formula becomes $\pi_{\text{old},x}(y_t)\cdot(1+\epsilon_c) = 1.02$. Since the probability output by the policy model cannot exceed 1, the upper clipping threshold of 1.02 is never reached. Thus, the gradient bias is nonnegative for this token, leading to a net positive gradient bias on the policy model, which leads to increase in probability on this token.

18.06 On the other hand, for a low-probability token where $\pi_{\text{old},x}(y_t) = 0.02$, the policy model receives a negative gradient bias when $\pi_{\theta,x}(y_t) > \pi_{\text{old},x}(y_t)\cdot(1+\epsilon_c) = 0.024$; and positive gradient bias when $\pi_{\theta,x}(y_t) < \pi_{\text{old},x}(y_t)\cdot(1-\epsilon_c) = 0.016$. The low threshold for negative gradient bias makes penalties more likely than in the high-probability case.

18.07 The clipping range width scales linearly with the original probability: higher-probability tokens have wider ranges and face fewer penalties. This asymmetric treatment prevents low-probability samples from receiving substantial upweighting during training, causing the model to concentrate probability mass on its existing distribution.

18.01 按符号取期望

分别加权正、负优势条件期望，再按R所在区域组合，得三段梯度。这里使用优势与相关梯度量独立的简化分析。

18.02 零均值恒等式

P(A≥0)E[A|A≥0]与P(A<0)E[A|A<0]大小相等、符号相反，因此区间内两者抵消。

18.03 精确系数

μ为正优势加权均值；非退化随机奖励时μ>0，常数奖励时为0。最终偏置是μ乘以分段的±□R，区间内为0。

18.04 方向解释

低于旧概率的(1-ε)倍时有恢复向上的偏置，高于(1+ε)倍时有向下偏置。该token局部方向不等于共享参数模型所有token的净变化。

18.05 B.1.3·先验解释

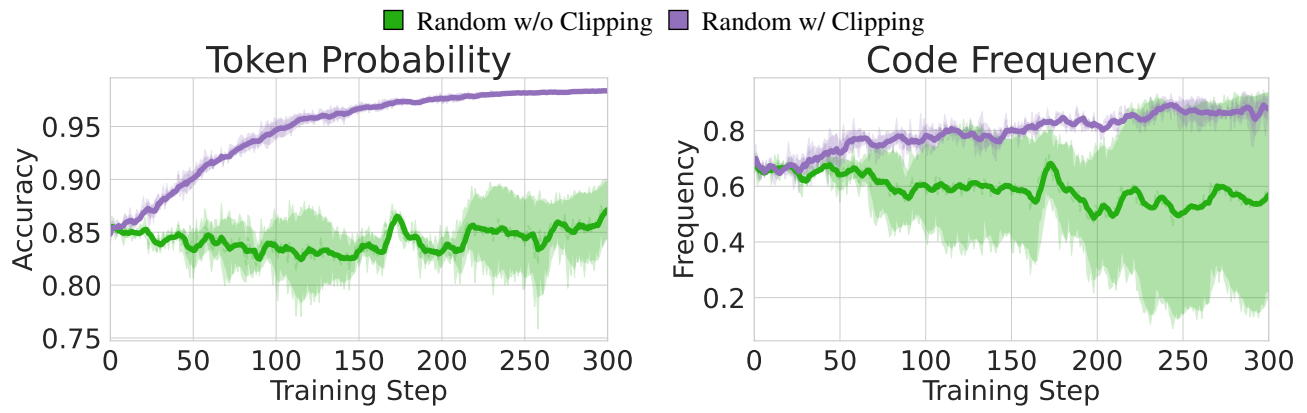
用概率边界例子说明高概率token与低概率token所受裁剪并不对称；这是支持机制的示例，尚非任意网络中全局行为的完整定理。

18.06 阈值例子

0.85的上界为1.02，不可达；0.02的上界0.024可达，下界0.016。概率尺度导致绝对裁剪宽度不同。

18.07 作者推论

裁剪可能限制低概率行为的大幅增权，使概率更集中于既有分布。需与跨种子消融及行为轨迹合看，而非只靠阈值例子推出全部涨分。



(a) Token Probability

(b) Code Frequency

19.01 – *Figure 9.* Token probability and code frequency of random rewards with (purple) and without (green) clipping on Qwen2.5-Math-7B.

19.02 – **Empirical validation.** We empirically validate our conjecture by disabling the clipping term in the GRPO loss, and observe differences in the training dynamics. Specifically, we focus on *Token Probability* π_θ and *Code Frequency* as defined below.

19.03 – Recall that π_{old} is the old policy from the previous training step and the π_θ the current policy, let G be the number of rollouts generated per prompt; let B be the mini batch size, which is the number of prompts on which a single gradient update is performed; and let M be the number of gradient updates from the old policy, π_{old}, to the new policy model, π_θ. Note that $G \cdot B \cdot M$ is the total number of rollouts generated by π_{old}. The average token probability of π_θ over all rollouts $\{y^{(k)}\}$ at any step can be expressed as:

$$\pi_{\theta,x}(y) = \frac{1}{M}\sum_{i=1}^{M}\frac{1}{B}\sum_{j=1}^{B}\frac{1}{G}\sum_{k=1}^{G}\pi_{\theta_i,x_j}(y^{(k)}).$$

19.04 – Figure 9 (a) shows that the average token probability mass increases for the standard loss of GRPO with clipping (purple line), but stays relatively constant when clipping is disabled. In addition, we log the frequency of code reasoning throughout the RLVR training in Figure 9 (b). We hypothesize that the increase in average token probability correlates with the increase in code reasoning behavior, which, as we conjectured in Section 5, enhances the performance of the model, as we empirically show in Figure 6 of Section 5.2.

19.05 – **Why only Qwen-Math models benefit.** The clipping bias mechanism operates on all models, but its effectiveness depends on what behaviors get amplified. The bias systematically favors a model's pre-existing high-probability behaviors, but only benefits performance if those behaviors correlate with correctness. For Qwen2.5-Math models, code reasoning occurs in 65% of responses and correlates strongly with correctness (64% accuracy vs. 29% without code). We observe that, when RLVR with clipping bias increases the model's code reasoning frequency to >90%, performance improves substantially. *No-Code models* (Llama, OLMo2-7B) generate no code reasoning, so clipping bias has no beneficial pattern to amplify. On the other hand, *Bad-Code models* (OLMo2-7B-SFT) generate code 98% of the time but with 21% accuracy vs. 40% for natural language, indicating that good performance does not rely on code reasoning. Overall, the clipping bias is universal, but its impact on performance depends entirely on whether the model's dominant pre-existing behaviors happen to be effective reasoning strategies.

19.06 – **Implications.** Our findings reveal that GRPO's clipping mechanism provides a meaningful training signal even from pure noisy rewards, by systematically favoring the model's pre-existing behavioral patterns. This suggests that the apparent "reward signal" in random reward training is actually an artifact of the optimization algorithm's bias toward exploiting learned priors rather than exploring new behaviors. This mechanism explains why random rewards work for Qwen2.5-Math models (which have strong code reasoning priors) but fail for other model families lacking these pre-existing capabilities. The training algorithm amplifies whatever reasoning patterns already correlate with correctness, regardless of the reward signal's actual informativeness.

19.01 图9·机制诊断

开启裁剪时，采样token平均概率与代码频率均上升；去掉裁剪则变化较小。两条轨迹为机制假说提供经验支持。

19.02 实施·观测变量

作者关闭裁剪比较训练动态，分别记录token概率和代码频率，尝试把梯度分析连接到可测的行为变化。

19.03 统计量定义

在旧策略生成的rollout上，跨更新、小批及回答求平均当前策略概率。它不是测试准确率，也不等价于整个词表分布的熵。

19.04 发现·相关链

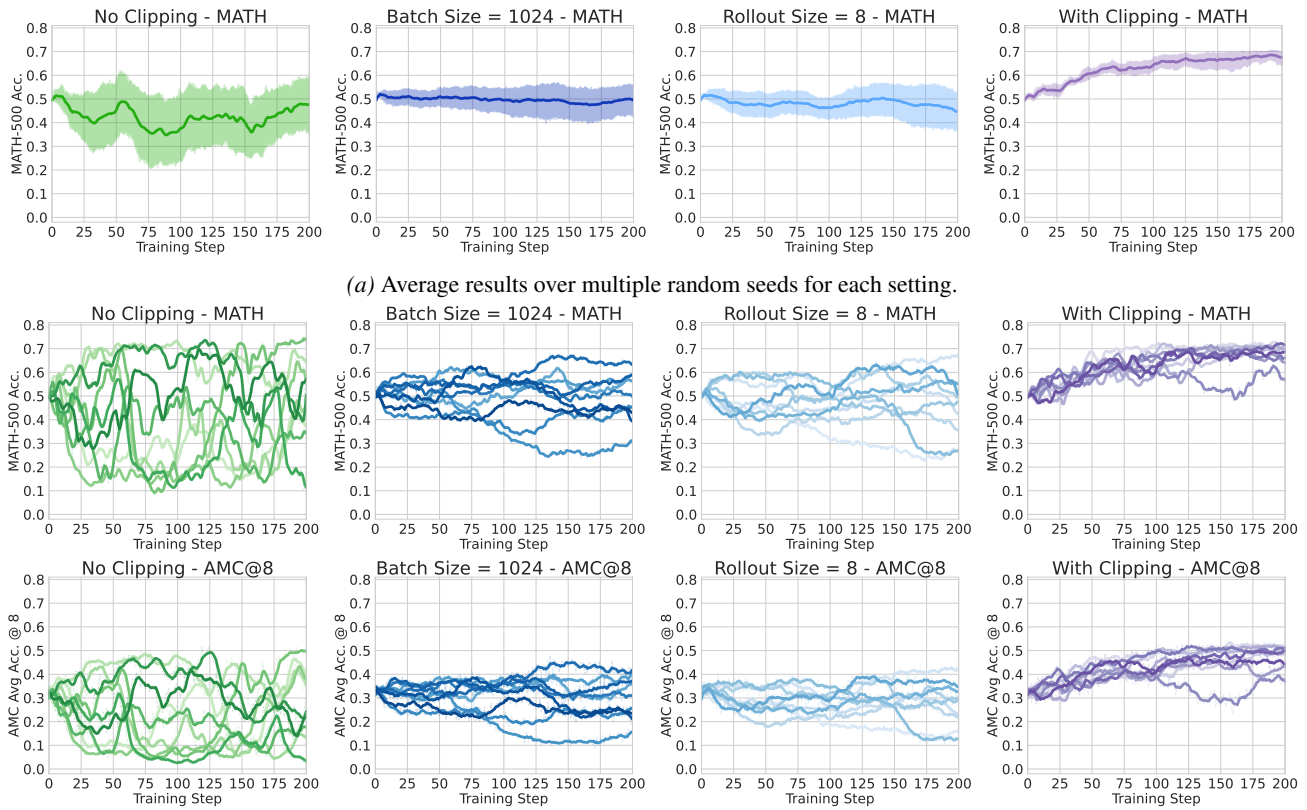
裁剪→平均token概率上升→代码频率上升→成绩提升，是作者提出的解释链；各箭头的证据强度不同，图中相关轨迹不能独立证明完整因果链。

19.05 模型差异解释

优化偏置可以存在于多模型，但只有放大的既有策略有用时成绩才会提升。无代码或低质量代码先验都可能无法受益。

19.06 含义

随机奖励并不携带数学正确性信息；可见收益可能来自优化偏置与已有行为的组合。作者的解释要放在其模型、奖励与超参范围内理解。



(a) Average results over multiple random seeds for each setting.

(b) Turning off clipping with 8 gradient updates per rollout.

(c) Increasing batch size to ensure only 1 gradient update per rollout.

(d) Decreasing rollout batch size to ensure only 1 gradient update

(e) Default GRPO loss with random reward.

Figure 10. RLVR performance on random rewards with disabled clipping across multiple different seeds. Models without clipping show no meaningful performance improvement on average, while models with clipping demonstrate consistent performance gains using random rewards. Disabling clipping in implementation produces high variance in results, occasionally yielding high accuracy scores. Experiments with adjusted batch sizes exhibit greater stability but involve 8 times fewer gradient updates than the disabled clipping experiments.

B.2. High Stochasticity in RLVR Without Clipping

In this section, we empirically show that training without clipping has stochastic training dynamics and can sometimes obtain performance improvement despite the expected gradient being zero. Following the experimental setup in Section 4, we disable the clipping effect through the following interventions: (i) removing clipping from the loss implementation, (ii) increasing mini-batch size to match rollout size, and (iii) decreasing the rollout size to match mini-batch size across multiple random seeds. We report the results in Figure 10 across different random seeds.

When clipping is disabled by adjusting batch size, model performance remains stable within a consistent range without converging toward either extreme of the accuracy spectrum. These runs involve 8 times fewer gradient updates due to the batch configuration.

In contrast, removing clipping from the implementation produces extreme stochasticity, occasionally resulting in high-performance convergence across multiple runs. While the exact mechanism remains unclear, we hypothesize that inherent randomness in RLVR training contributes to these observations. The group size $G = 16$ used in GRPO training creates high variance in gradient updates. Combined with the instability of unclipped training, this variance may accidentally reinforce certain reasoning patterns, leading to improved performance in some runs. We leave a systematic analysis of this behavior to future work.

20.01 图10·多种子

看均值与各次运行：直接去掉裁剪方差很大，偶尔也会高分；改批量的两个对照更稳定，但梯度更新次数少8倍。

20.02 B.2·为什么零期望还会涨

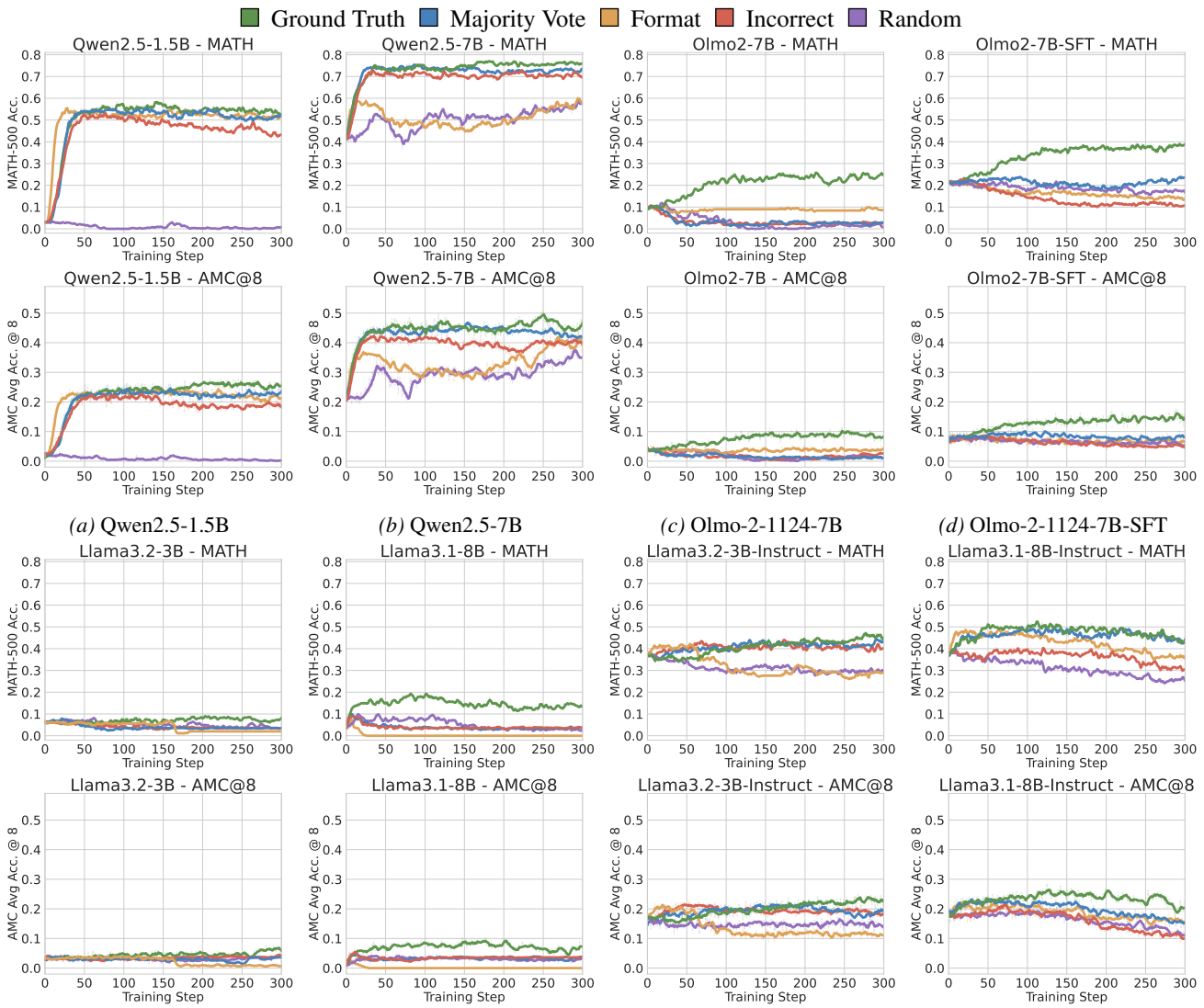
理论的总体期望为0不排除随机游走、有限样本噪声与非线性优化导致的个别成绩上升。

20.03 批量干预

调整batch后通常在原水平附近波动，没有系统性增长；同时改变优化预算，因此不能把全部差异只归因于裁剪函数。

20.04 直接删裁剪

高方差偶尔偶然强化有用策略；作者将组大小16与更新噪声作为可能解释，明确把系统分析留给后续工作。



(a) Qwen2.5-1.5B *(b)* Qwen2.5-7B *(c)* Olmo-2-1124-7B *(d)* Olmo-2-1124-7B-SFT

(e) Llama3.2-3B *(f)* Llama3.1-8B *(g)* Llama3.2-3B-Instruct *(h)* Llama3.1-8B-Instruct

21.01 *Figure 11.* Varying rewards across additional model classes. Spurious rewards remain effective on general-purpose Qwen2.5 models but generally fail to yield any gains on other model families. The performance improvements on non-Qwen2.5 models are substantially smaller compared to those observed in the Qwen2.5 family.

C. Additional Results on AMC

21.02 We supplement additional AMC results for non-Qwen models in Figure 11. The trends are consistent with the MATH-500 results that are shown in Section 3.

D. Additional Results on AIME 2024 and AIME 2025

21.03 We present additional results on the AIME benchmarks, which are challenging math Olympiad tests containing significantly harder problems than those in MATH-500 or AMC. We evaluate average@8 accuracy on AIME24 and AIME25 (Li et al., 2024). AIME25 was created after the release date of all models considered in our study. Thus, evaluating performance on AIME25 allows us to control the risk that our models' have seen similar problems during web pretraining. We evaluate the trained models from Section 2 and Section 3. We show results on Qwen2.5-Math models in Figure 12 and on the 8 additional models from Section 3 in Figure 13.

21.01 图11·AMC补充

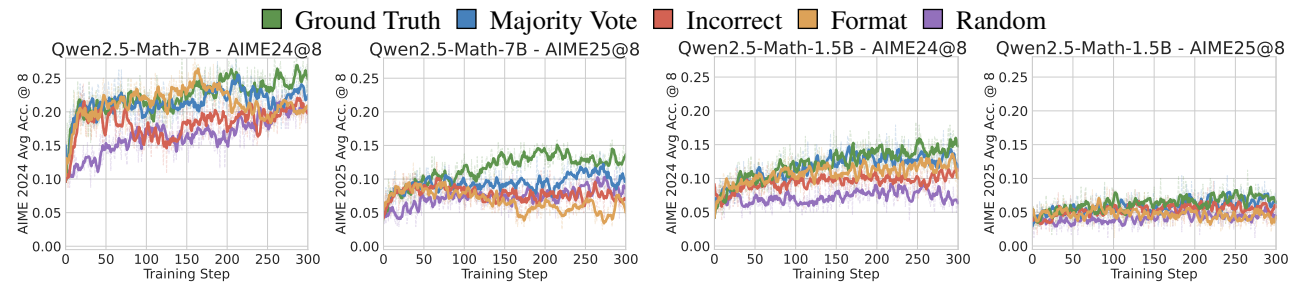
跨8个模型比较奖励，整体复现MATH上的家族差异。观察同一模型的起点和增量，而不只比不同模型曲线高度。

21.02 附录C·结论

AMC并未推翻主文：Qwen对弱/虚假信号更容易获益，非Qwen通常收益有限。

21.03 附录D·时间外评测

AIME25晚于这些模型发布，降低直接见过原题的风险，但不能据此排除全部相似题/训练分布因素；此处仍用average@8。



(a) Qwen2.5-Math-7B *(b)* Qwen2.5-Math-1.5B

22.01 – *Figure 12.* Qwen2.5-Math Model performance on AIME 2024 and AIME 2025.

D.1. Spurious Rewards Yield Significant RLVR Gains on Qwen2.5-Math

22.02 – As shown in Figure 12, spurious rewards can consistently yield performance gains on Qwen-Math models on AIME24. Intriguingly, we find that any AIME24 gains achievable from training Qwen models with spurious rewards largely vanish when evaluating on AIME 2025. We speculate that AIME25 contains questions that are more out-of-distribution to Qwen's pretrained knowledge; spurious rewards—which largely serve to elicit existing knowledge—hence no longer provide benefit.

D.2. (Lack of) Generalization to Other Models

22.03 – Overall, results on other models are largely consistent with our earlier findings on AMC and MATH-500 (§3). Only Qwen2.5-7B, Qwen2.5-1.5B, and Llama3.1-8B-Instruct exhibit any notable gains from any reward signals on AIME. For Qwen2.5 models, weak and spurious rewards can yield gains; for example, format reward for Qwen2.5-1.5B and incorrect reward for Qwen2.5-7B. For Llama3.1-8B-Instruct, only standard rewards (e.g., ground truth and majority vote) yield gains. As observed above, performance and gains from RLVR training are lower across the board for all models on AIME25.

E. Examination of Existing Methods on Models Beyond Qwen

22.04 – In this section, we examine the existing methods—TTRL and 1-shot RL—that apply weak supervision during RL, as shown in Figure 15. We find both methods are significantly effective on Qwen models, but not others.

F. Compound Rewards that Inhibit Code Reasoning

22.05 – **Inhibiting code reasoning during RLVR with spurious rewards can reduce gains on Qwen2.5-Math-7B, but increase gains on other models.** We hypothesized that code reasoning is part of the source of weak and spurious rewards gains. Contrapositively, *penalizing* code frequency could potentially reduce the gains of these rewards in Qwen2.5-Math. To
22.06 – test this, we designed compound rewards that intersect each weak or spurious reward with a *no Python reward*, so that a response is rewarded if and only if (1) it satisfies the original spurious reward condition and (2) it does not contain the string "`python`". Consistent with our hypothesis, format rewards cease to improve Qwen2.5-Math-7B when paired with the no code reward (Figure 16c). The spurious incorrect compound reward matches the original incorrect reward on MATH-500; on more difficult benchmarks such as AMC and AIME, gains also persist but are reduced (Figures 16b, 14). Thus, while ablating code reasoning does hurt the incorrect reward as predicted, we posit that other beneficial behaviors (e.g., reduced repetition, see Appendix G) may still be superficially elicited. In addition, ground-truth rewards still yield gains, suggesting benefits beyond eliciting code reasoning (consistent with the code-frequency trends in Figure 6).

22.07 – Intriguingly, for *Bad-Code* models Qwen2.5-7B and OLMo2-7B-SFT, compound rewards often *outperform* the originals. Most strikingly, OLMo2-7B-SFT—which degrades under standalone format or incorrect rewards—gains +8.9 and +5.5 points, respectively, once the no-code reward is added. We hypothesize that this is because Qwen2.5-7B and OLMo2-7B-SFT exhibit weak code reasoning before RLVR training, so compound rewards explicitly downweight a behavior that is suboptimal for these models.

22.08 – In Figure 14, we present additional results for our compound rewards on (1) more models (Qwen2.5-1.5B, Qwen2.5-Math-1.5B) and (2) more benchmarks (AIME24, AIME25). Overall, results corroborate our analysis in Section 5.3; Qwen2.5-Math-1.5B follows the same overall trends as Qwen2.5-Math-7B. Compound rewards also continue to benefit

22.01 图12·更难任务

Math模型在AIME24上可从虚假奖励获益，但在AIME25上优势明显减弱；各年份难度不同，不能把差异全归结为污染或记忆。

22.02 D.1·作者解释

猜测AIME25更偏离预训练分布，使激发已有知识的奖励较难奏效。它是分布外解释，不是已验证的数据污染结论。

22.03 D.2·外推

部分Qwen仍能从弱信号获益，Llama的收益更多来自真实/多数票监督。AIME每年仅30题，小幅差异需谨慎看待。

22.04 附录E·既有方法

复测TTRL和单样本RL，结果同样呈现模型依赖；结合A.7的设置差异判断，不宜据此否定原方法所有配置。

22.05 附录F·反向干预

复合奖励=原奖励条件且回答不含python。抑制Math模型代码常削弱增益，用与正向诱导相反的干预检验机制。

22.06 F·结果细分

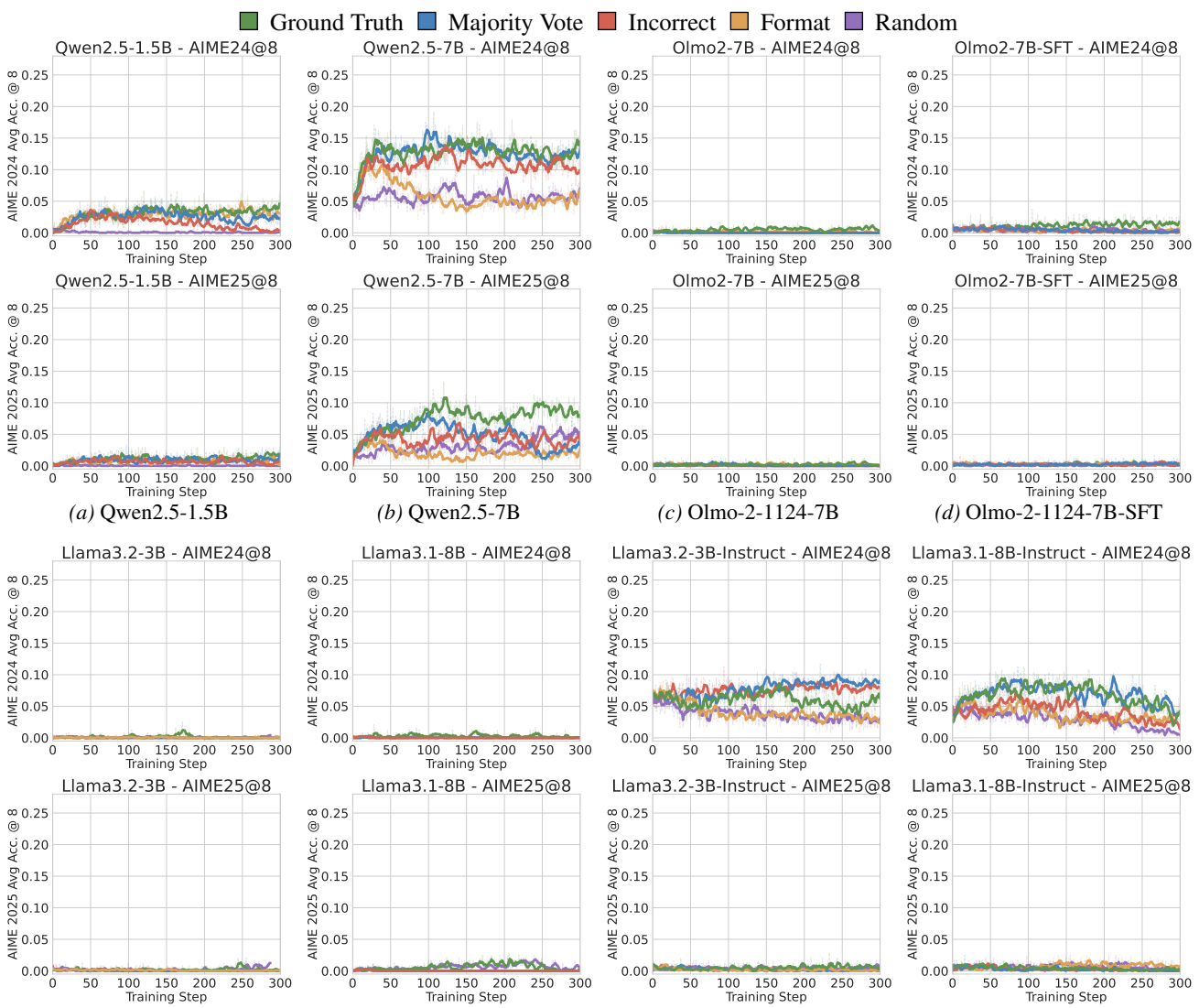
格式收益可消失；错误奖励在MATH仍可保留收益，在更难任务上减弱；真实奖励依然有效。代码因而只是部分机制。

22.07 F·相反结果

Bad-Code模型减少低质量代码反而涨分，OLMo-SFT复合格式/错误奖励约提升8.9/5.5点，支持“选对该模型擅长的策略”这一解释。

22.08 F·补充对照

更小模型及AMC/AIME大体支持此趋势，但存在尺寸差异，下一页继续讨论尚未解释的例外。



(a) Qwen2.5-1.5B *(b)* Qwen2.5-7B *(c)* Olmo-2-1124-7B *(d)* Olmo-2-1124-7B-SFT

(e) Llama3.2-3B *(f)* Llama3.1-8B *(g)* Llama3.2-3B-Instruct *(h)* Llama3.1-8B-Instruct

23.01 *Figure 13.* Varying rewards across additional model classes on the AIME 2024 and AIME 2025 benchmarks. Note that AIME 2025 was released after all the models' release dates, serving as a good resource to examine any dataset contamination phenomenon. Weak and spurious rewards (except for random) remain effective on general-purpose Qwen2.5 models but generally fail to yield any gains on other model families. The performance improvements on non-Qwen2.5 models are substantially smaller compared to those observed in the Qwen2.5 family. Note that the AIME benchmarks contain 30 questions each, so small differences in accuracy (less than ~ 2 pp.) may not be significant.

23.02 Qwen2.5-1.5B, where gains are comparable to the gains from using the original reward. This slightly contrasts our observations in Qwen2.5-7B, where compound rewards often yielded stronger gains. We conjecture that Qwen2.5-1.5B is stronger at code reasoning, which is consistent with our finding in Section 5.3 that inducing code reasoning via prompting improves performance in in Qwen2.5-1.5B but not Qwen2.5-7B. We are unsure of the exact reason for this discrepancy between these two models (which have the same pretraining data) and leave it for future work.

G. Beyond Code Reasoning: Another Beneficial Pattern that RLVR Can Easily Elicit

23.03 In the main paper, we show that RLVR with spurious rewards can improve Qwen2.5-Math's performance by surfacing useful reasoning patterns learned during pretraining, and we use code reasoning as a standout example. We note that code reasoning is one distinctive reasoning representation, but not the only one. In this section, we briefly discuss another pattern—no repetition—that can also be easily elicited by RLVR in addition to code reasoning.

23.01 图13·其他模型AIME

弱/虚假奖励的收益主要仍在Qwen；小测试集下小于约2个百分点的差异可能不显著。无置信区间时不要过度解读曲线细微差别。

23.02 F续·例外

通用1.5B的复合奖励收益与原奖励相近，7B则可能更受益。作者承认即使共享预训练数据，尺寸差异的准确原因仍未知。

23.03 附录G·另一种行为

代码推理不是唯一易激发模式；减少重复输出也可能提高成绩，用它检验更一般的行为激发假说。

24.01 We observe that Qwen2.5-Math models have a relatively higher tendency to produce repetitive outputs compared to Llama3 and Olmo2 models. We find that answers with code reasoning often do not have this issue. Therefore, we further study the effect of purely discouraging repetition in answers. To study this, we design a new *repetition reward* that returns a score of 0 when the answer contains any string repeated more than 10 times. Otherwise, it rewards the model with a full score of 1.

24.02 As shown in Figure 17, the RLVR no-repetition reward improves the performance of Qwen2.5-Math-7B and Qwen2.5-Math-1.5B on MATH and AMC. We observe minimal or even negative improvement on other models. Based on our findings, we hypothesize that various patterns exist whose presence correlates with answer correctness. These patterns, including code reasoning and no repetition, can be easily elicited by RLVR even when the rewards provide no information about the ground-truth answers. Still, the effectiveness of eliciting these patterns is heavily model-dependent.

H. Switch of Reasoning Strategies during RLVR

24.03 In this section, we present further analysis on the change of reasoning strategies during RLVR for Qwen2.5-Math and Qwen2.5 models.

24.04 **Reasoning strategy switches during RLVR.** Qwen2.5-Math-7B's accuracy increases by an average of 23.5 absolute points across different training signals. To further break down this gain, we track the performance of models trained on each training signal across four disjoint subsets of the test prompts: **(1) Code→Code**: the model uses code reasoning both before and after RLVR; **(2) Code→Lang**: the model initially uses code reasoning but switches to natural language reasoning; **(3) Lang→Code**: the model initially uses natural language reasoning but switches to code reasoning; and **(4) Lang→Lang**: the model uses natural language reasoning both before and after RLVR. Specifically, we focus on two interconnected metrics:
24.05 frequency and accuracy of each subset. To systematically quantify the contribution of each subset to the performance gain, we define a *Partial Contribution Score*, C_d, for any subset $d \subseteq \mathcal{D}$ of the entire test set $\mathcal{D}$, such that C_d is the ratio between the net increase in the number of correctly answered problems in d divided by the net increase in the number of correctly answered problems in $\mathcal{D}$:

$$C_d = \frac{\sum_{x \in d} \mathbb{I}[\text{correct}(x_t)] - \mathbb{I}[\text{correct}(x_0)]}{\sum_{x \in \mathcal{D}} \mathbb{I}[\text{correct}(x_t)] - \mathbb{I}[\text{correct}(x_0)]}, \quad \text{where } x_0 \text{ and } x_t \text{ are the initial and final answers.}$$

Frequency: Figure 18 shows Qwen2.5-Math-7B's reasoning strategy switching pattern. For all weak and spurious rewards, the model uses more code reasoning after RLVR. While few originally-code-reasoning problems switch to language reasoning (Code→Lang), most originally-language-reasoning problems convert to code reasoning (Lang→Code). Ground truth reward does not follow this pattern. For *Bad-Code* models (Qwen2.5-7B and OLMo2-7B-SFT), meaningful rewards steer models away from bad code reasoning. Code reasoning decreases with ground truth, majority vote, and incorrect rewards for Qwen2.5-7B, but only with ground truth and majority vote for OLMo2-7B-SFT (Figures 6b and 6c). For *No-Code* models, RLVR fails to elicit meaningful changes in reasoning strategy, as this capability is likely not learned during pre-training.

24.07 *Table 3.* Partial contribution to the overall performance gain averaged over rewards that successfully steered the model's reasoning strategy (Figure 6).

Model	Qwen2.5-Math-7B	Qwen2.5-Math-1.5B	Qwen2.5-7B
Avg. Total Gain	↑ 23.5%	↑ 28.5%	↑ 30.6%
$\mathbf{C}_{\text{Code}\to\text{Code}}$	11.6%	2.8%	0.2%
$\mathbf{C}_{\text{Code}\to\text{Lang}}$	8.6%	2.0%	**93.9%**
$\mathbf{C}_{\text{Lang}\to\text{Code}}$	**58.3%**	**78.7%**	0.0%
$\mathbf{C}_{\text{Lang}\to\text{Lang}}$	21.4%	16.5%	5.9%

Accuracy: From Figure 18, there is a drastic increase in accuracy in the Lang→Code subset after RLVR across all training signals. This is reflected in Table 3, which shows that 58.3% of the performance gain of Qwen2.5-Math-7B is from this subset. Similarly in Qwen2.5-Math-1.5B, switching from natural language reasoning to code reasoning contribute to 78.7% of the performance gain. For the *Bad-Code* models, Code→Lang contributes to 93.9% of the performance gain of Qwen2.5-7B. This is intuitive, as the model has a higher langauge reasoning accuracy than code reasoning accuracy, RLVR training essentially encourages the model to use the reasoning strategy that it is better at. For *No-Code* models, since there is

24.01 G·无重复奖励

回答中任意字符串重复超过10次给0，否则给1。此信号不验证答案，却可减少退化式重复，改善可用输出比例。

24.02 G·发现

无重复奖励能提高Qwen-Math，但其他模型收益很少或为负；说明“有益行为”及其提升空间都依赖基模。

24.03 附录H·分析目标

进一步追踪训练前后策略转换，区分同一种策略内部变准与改用另一种策略带来的变化。

24.04 H·四组划分

按每道题训练前/后是否用代码分为C→C、C→L、L→C、L→L；四组互斥，覆盖整个测试集。

24.05 贡献分数

某组“新增答对题数”除以全测试集“新增答对题数”。它是净收益分解，不是该组准确率，也不直接证明因果；分母很小时不稳定。

24.06 H·频率

Math模型在弱/虚假奖励下大量由语言转代码；Bad-Code模型在较有效监督下则趋向减少代码，No-Code模型策略变化较少。

24.07 表3·贡献

Math-7B约58.3%净增益来自语言→代码，Math-1.5B约78.7%；通用Qwen-7B约93.9%来自代码→语言。不是所有模型“写更多代码就更好”。

24.08 H·准确率

主要转换组准确率提高，支持策略重分配的重要性；同策略组也有贡献，所以不能把收益全等同于策略切换。

Prompt Name	System Prompt	User Prompt
Qwen Default (Yang et al., 2024a)	Please reason step by step, and put your final answer within `\boxed{}`.	{}
Math Problem	You are a helpful Assistant.	Math Problem: {}
Simplerl-zoo (Zeng et al., 2025)	You are a helpful Assistant.	{}\nPlease reason step by step, and put your final answer within `\boxed{}`.
Sober (Hochlehnert et al., 2025)	Please reason step by step, and put your final answer within `\boxed{}`. [6]	Solve the following math problem efficiently and clearly. The last line of your response should be of the following format: 'Therefore, the final answer is: The last line of your response should be of the following format: 'Therefore, the final answer is: `\boxed{ANSWER}`. I hope it is correct' (without quotes) where ANSWER is just the final number or expression that solves the problem. Think step by step before answering.\n\n{}
Spurious Prompt	Lorem ipsum dolor sit amet, consectetuer adipiscing elit.	Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat.\n\n{}

Table 4. Details of different prompts used in previous RLVR research. We explore the impact of existing prompts (Qwen Default, Simplerl-zoo, and Sober) and 2 our two proposed prompts—the first prompt, Math Prompt, indicates the evaluation domain without extra information about the format; the second prompt, Spurious Prompt, is a randomly picked LaTeX placeholder text generated by LIPSUM. Our motivation for introducing these 2 prompts is to study how concise domain knowledge or even random strings in context can boost the evaluation performance.

no code reasoning before or after RLVR, all performance gains (or losses) are from the **Lang→Lang** subset. These results suggest that much of the accuracy gain from RLVR on these spurious rewards in Qwen2.5-Math and Qwen2.5 is simply from eliciting the right reasoning strategy from the model.

I. Spurious Prompts: Qwen2.5-Math-7B's Unreasonably High Sensitivity to Prompts

As we show in Section 5.3, prompting the model to use code reasoning brings a 15.0% gain on Qwen2.5-Math-7B. This suggests that prompt engineering is a valid approach to elicit desired behaviors, which does not require parameter updates. In this section, we conduct an additional analysis on how choosing different existing prompts can impact model behavior and how such a technique interacts with RLVR. In addition, we show that model performance can be improved even when using some task-unrelated, information-less prompts, which we name "spurious prompts."

I.1. Existing Prompts

We collect different prompts from popular RL training frameworks and recent evaluation works: Qwen Default (Yang et al., 2024a), SimpleRL-Zoo (Zeng et al., 2025), Sober Reasoning (Hochlehnert et al., 2025), and our hand-constructed prompts (detailed in Table 4). Specifically, we introduce two new prompts—the first prompt, Math Prompt, indicates the evaluation domain without extra information about the format; the second prompt, Spurious Prompt, is a randomly picked LaTeX placeholder text generated by LIPSUM. Although it has been known that LLM evaluation can be sensitive to evaluation prompts (Sclar et al., 2024), our motivation to introduce these two particular prompts is to study how concise domain knowledge or even random perturbation in context can impact evaluation performance. We perform RL training using these

25.01 表4·提示对照

比较默认、领域提示、既有模板及无关LIPSUM文本。表内英文指令是论文实验材料，不是本次任务指令；关注模板如何改变评测起点。

25.02 H续·结论

No-Code模型收益只能来自语言→语言组。策略切换解释了相当一部分增益，但不同模型应选择不同有效策略。

25.03 附录I·问题

提示可在不更新参数时带来明显提升，作者进一步考察提示效应与RL收益是否重叠，避免把起点差异误算成训练效果。

25.04 I.1·实验设计

领域提示只给任务线索；SpuriousPrompt用无关占位文本。训练rollout和评测均使用所比较模板，保证每条轨迹口径一致。

	Default	MathProblem	SimpleRL-Zoo	Sober	Spurious
MATH Acc.	49.4	55.8	63.2	61.60	**68.8**
% Parsable	78.9	72.1	85.4	**93.1**	84.1

Table 5. Accuracy on MATH-500 and percent of parsable (format-following) responses on Qwen2.5-Math-7B with various prompts from Table 4. Even with a spurious prompt, the model is able to follow the format 84.1% of the time. It is not obvious that much of the performance can be explained by format-following.

prompts for both trajectory rollouts and evaluation.

As shown in Table 5, Sober prompt brings the highest parsable rate of the answers, while our spurious prompt leads to the highest accuracy on MATH-500. The results indicate that the model is very sensitive to prompts and the best performance does not necessarily require the highest parsable rate nor task-relevant information in context.

We further show the training trajectories using different prompts in Figure 19. We find that models trained with the Qwen default, MATH PROBLEM:, and Simplerl-zoo converge to similar performance after RLVR with the same training setting. We conjecture that the behavior that can be elicited by prompting tends to be a subset of the easy-to-elicit behaviors in RLVR, while RLVR can elicit additional behaviors that are not predefined in prompts.

I.2. Spurious Prompts

As Qwen2.5-Math-7B shows high sensitivity to the prompts, we are curious about how much it could be impacted by spuriously unrelated prompts, e.g., the placeholder text in LaTex generated by LIPSUM. Specifically, we construct a spurious prompt using LIPSUM with a similar prompt length as the Sober prompt in Table 4. Surprisingly, the spurious prompt gives the highest initial performance compared with the other commonly used task-specific prompts, as shown in Figure 19e. Although we find that the improvement does not always happen for randomly picked prompts, the high performance with our spurious prompt indicates that the model can be very sensitive to in-context perturbations, where the benefit may sometimes originate from this sensitivity rather than from the task-relevant content in the prompt.

J. Additional Results on Models That Have Undergone RL Training

In this section, we provide additional results for models that have been trained with reinforcement learning, including two Qwen Instruct models (Qwen2.5-Math-7B-Instruct (Yang et al., 2024a) and Qwen2.5-7B-Instruct (Yang et al., 2024b)) and one Tulu3 model (Llama-3.1-Tulu-3-8B (Lambert et al., 2024)). In this section, we show that the post-RL models may behave differently from other base or instruction-tuned models in RLVR.

As shown in Figure 20, Qwen Instruct models show minimal improvement from RLVR training across different reward types in our setup, even when using ground-truth rewards. We observe this same pattern in other models: their performance plateaus after sufficient RLVR training steps. Therefore, we conjecture that Qwen2.5-Math-7B-Instruct benefits less from our RLVR experiments because it has already reached saturation from its previous RL training. While improved RLVR algorithms or higher-quality training data could still yield gains on the currently saturated models, we leave this exploration for future work.

Furthermore, we examine the RLVR behavior of Llama-3.1-Tulu-3-8B (Figure 20c) and find that, unlike the Qwen2.5 Instruct models, training with ground truth rewards provides only marginal gains on MATH and limited gains on ACM and AIME. The trends we observe for Llama-3.1-Tulu-3-8B are similar to those we observe for its base model Llama-3.1-8B. We conjecture that this might be because these models share similar prior knowledge before RL training, which leads to similar RLVR behaviors. However, unlike the Qwen Instruct models, Llama-3.1-Tulu-3-8B still shows gains on MATH despite having been trained with RLVR on the Tulu3 dataset.

K. Qualitative Analysis on Qwen2.5-Math-7B's Coding Reasoning Behaviors

In this section, we show several qualitative examples on how Qwen2.5-Math-7B can reason in code. In addition, we show its code reasoning behavior is robust to numerical perturbations—the model generates similar code to solve the numerical perturbed question. However, we find Qwen2.5-Math-7B only uses natural language solutions when we rephrased the question using an alternative narrative.

26.01 表5·解析率不等于成绩

Sober解析率最高93.1%，但无关提示准确率最高68.8%；更易解析能解释部分收益，不能解释全部排名。

26.02 提示·发现

同一模型准确率对上下文高度敏感；“提示有任务信息”与“性能更高”不是一一对应关系。

26.03 训练·趋同

若干模板经RL后趋向相近成绩。作者猜提示激发的行为是RL易激发行为的子集；这是机制假说，不能从曲线趋同直接证明集合包含。

26.04 I.2·无关提示

选取长度接近Sober的LIPSUM文本得到高初始分；作者同时指出随机选择提示并非总能提高成绩，避免把单个成功例推广为通则。

26.05 附录J·已做RL模型

加入两个Qwen指令模型与Tulu3，考察既往RL是否影响进一步训练收益。

26.06 J·饱和假说

本设置下Qwen指令模型即便真实奖励也增益很小，作者猜已趋饱和；不能等同于模型能力的绝对上限，更好的算法/数据仍可能有效。

26.07 J·另一家族

Tulu3仍可从真实奖励获益，行为更接近其Llama基模。做过RL并不自动意味着所有模型都呈相同饱和现象。

26.08 附录K·定性目标

通过数字扰动与叙述改写看策略是否变化；少量案例有助提出假说，不能代替系统泛化或污染检验。

27.01 **Qwen2.5-Math-7B can reason in code.** In Figure 22–23, we show 3 qualitative examples of Qwen2.5-Math-7B outputs on 3 randomly picked questions from MATH-500. We find Qwen2.5-Math-7B is able to conduct coding reasoning, i.e., solving a problem by writing code, and predict the code execution outputs. Surprisingly, the model can predict the code execution outputs with a relatively high accuracy—in the examples shown in Figure 22 and Figure 21, Qwen2.5-Math-7B is able to compute the execution answers with 16-float precision **without** access to any code interpreter. In the example shown in Figure 23, Qwen2.5-Math-7B gives a wrong answer 4323. However, it is still very close to the ground-truth answer 4343.

27.02 **Qwen2.5-Math-7B's code reasoning behavior is robust to numerical perturbations.** We further perturbed the number 999 in the question shown in Figure 23 to a few random integer numbers between the range of 100 and 1000, as shown in Figure 24–26. We find that Qwen2.5-Math-7B is capable of solving our perturbed questions with similar code snippets in their answers. For example, in Figure 24, we show that Qwen2.5-Math-7B is able to derive the correct answer 4344 in the numerically perturbed version of the question in Figure 23—we changed the input number from 999 to 1000, and the corresponding ground-truth answer changed from 4343_6 to 4344_6. We find Qwen2.5-Math-7B used the same code function to solve the question, and interestingly, it predicts the correct answer for the perturbed question, although it predicted an inaccurate answer for the original question in Figure 23.

27.03 **Qwen2.5-Math-7B's code reasoning behavior may not directly generalize to rephrased questions.** We also rephrased the question in Figure 23, Figure 25, and Figure 26, respectively, with an alternative narrative. As shown in Figure 27–29, Qwen2.5-Math-7B first identifies and concludes the same core question in the first sentence of its answer. However, it does not use any code to answer this rephrased question. We hypothesize that whether Qwen2.5-Math-7B uses code reasoning to answer the question depends on the specific prompt distribution, i.e., whether the question is close to any of its training data that is augmented with code solutions, rather than the problem itself.

27.01 K·代码与结果均为生成

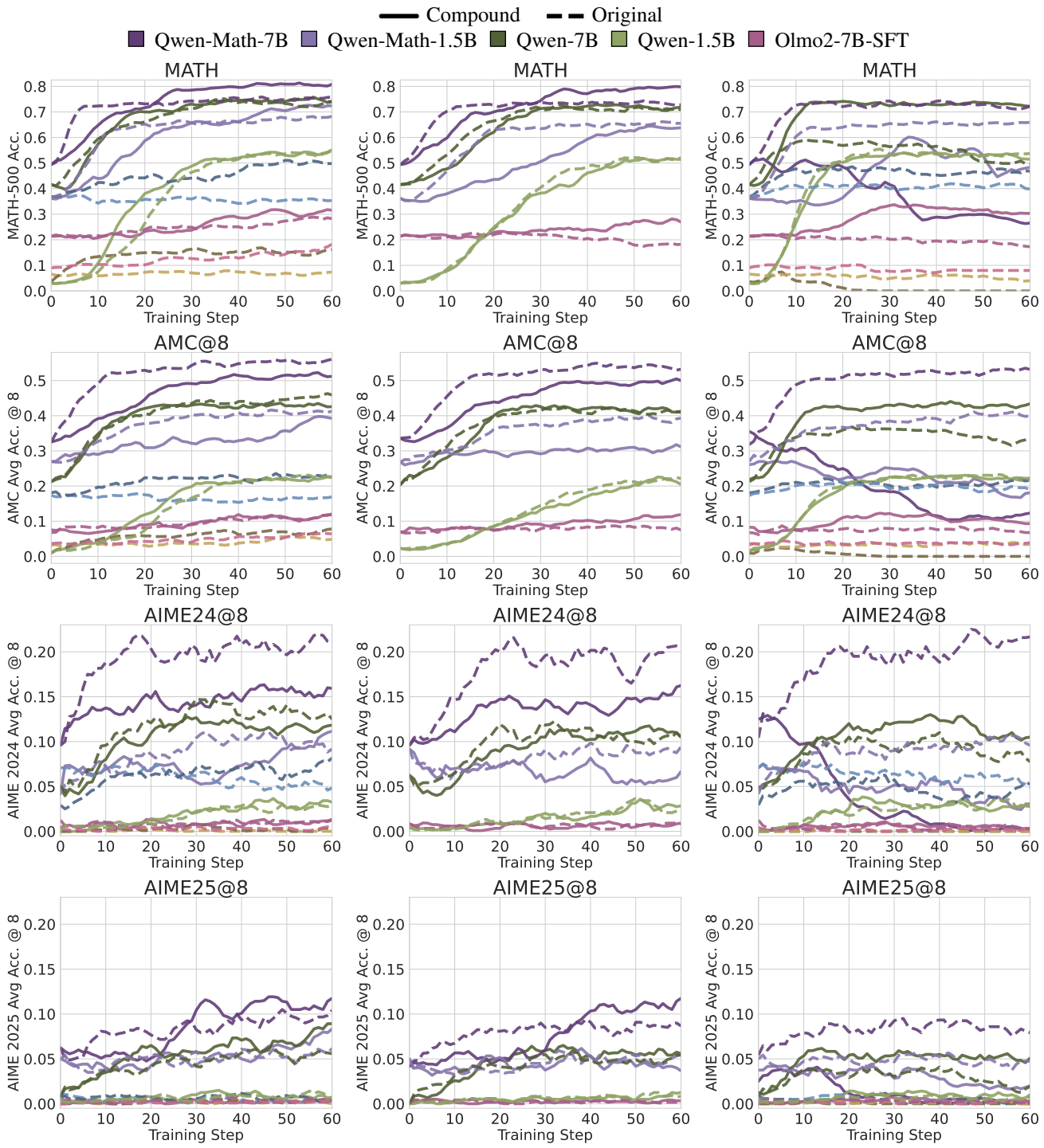
部分例子能给出很多位小数，但没有执行器。另有999进制转换输出错误，说明生成像执行日志的文本不等于可靠执行。

27.02 K·数字扰动

把999改为1000、998或100，仍生成相似除基取余代码且答案正确；原题却答错，显示代码模板与模拟结果可靠性可以分离。

27.03 K·改写敏感

相同数学问题换叙述后改用自然语言。作者推测策略与训练提示分布相关；此证据不能单独证明或排除记忆、数据污染。



(a) Ground Truth w/o Python *(b)* Incorrect w/o Python *(c)* Format w/o Python

28.01 *Figure 14.* We present additional compound reward results on (1) smaller 1.5B Qwen models and (2) AMC and AIME benchmarks. See Section 5.3 for full details of the setup. Our compound rewards intersect (a) our original rewards with a (b) *no Python reward* that only 28.02 rewards responses without Python code. Overall, our findings here are largely consistent with those from our main text. Note that AIME is a very small test set, so small differences in accuracy (less than $\sim$ 2 percentage points) may not be meaningful.

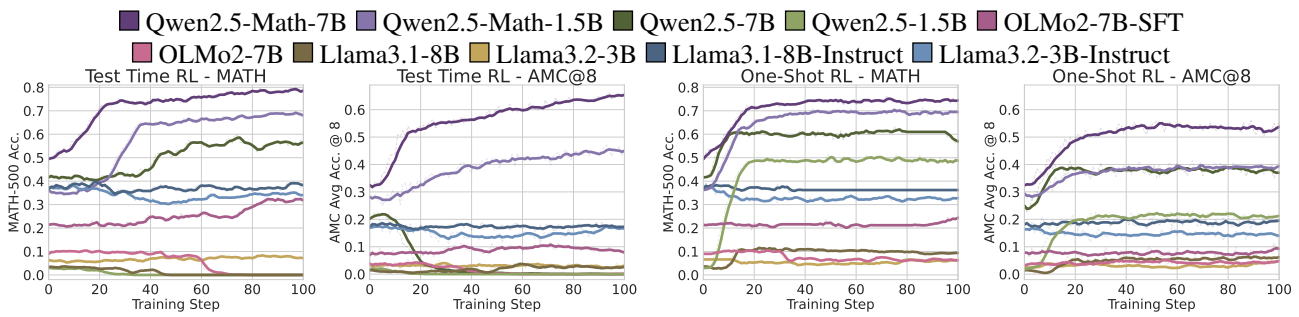
28.01 图14·复合奖励补充

每个模型比较原奖励与“原条件且无python”的奖励，跨MATH/AMC/AIME观察抑制代码后的变化；它改变行为选择，也改变得奖稀疏度。

28.02 读图·尺寸与任务

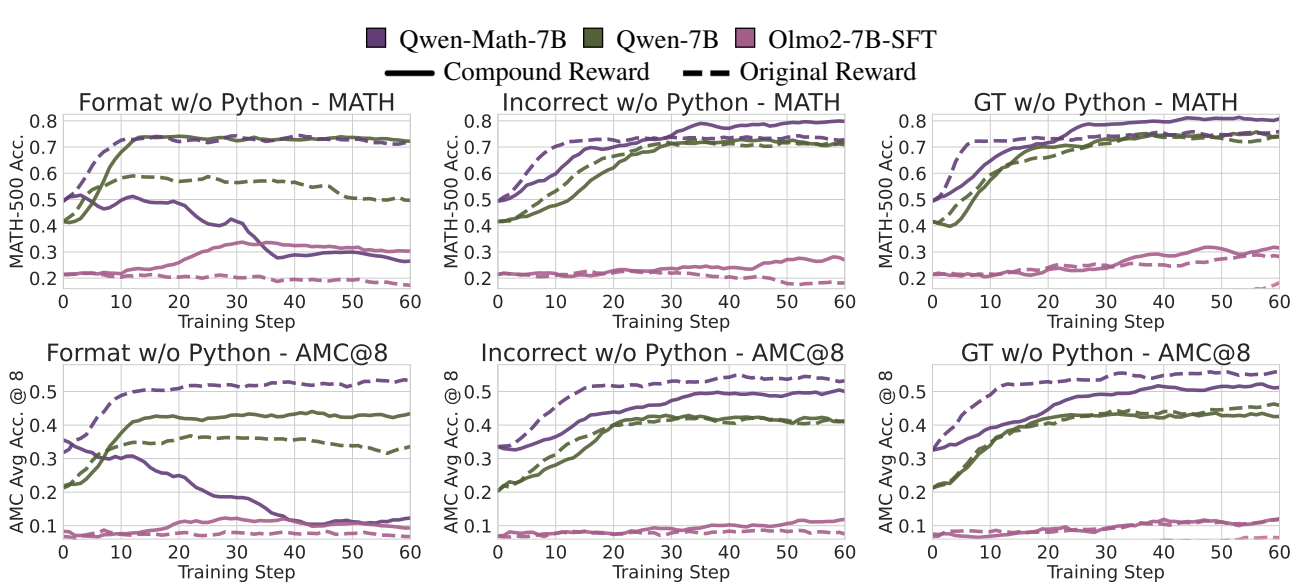
Math模型常因抑制代码损失收益，通用模型未必。AIME仅30题，小幅曲线差异不能据此排列方法优劣。

蓝色数字对应段落 / 图表；术语在首次出现处批注



(a) TTRL Results *(b)* One-Shot RL Results

29.01 *Figure 15.* We evaluate two recent weak supervision RL methods—TTRL (Zuo et al., 2025) and One-Shot RL (Wang et al., 2025b)—on diverse base models. We find that the proposed training rewards can consistently work on Qwen models. Yet with few exceptions, those same proposed signals often yield no gains on other model families, mirroring the limited generalization observed when training with our own spurious rewards. See Appendix A.7 for setup details.



(a) Format w/o Python *(b)* Incorrect w/o Python *(c)* Ground Truth w/o Python

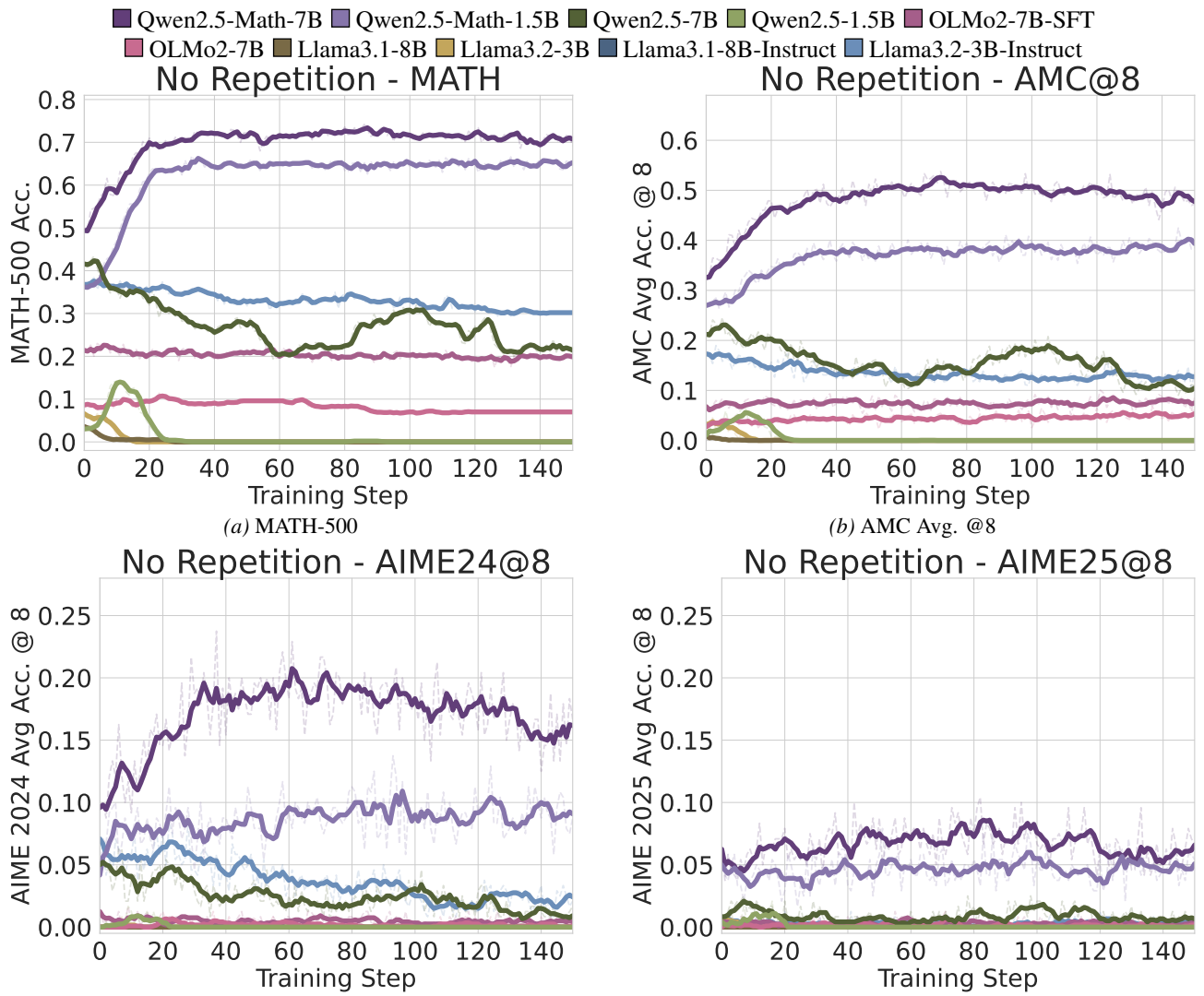
29.02 *Figure 16.* RLVR with compound rewards that intersect (i) our original rewards with a (ii) *no Python reward* that only rewards responses without Python code. We defer corroborating results on AIME and more models to Appendix F.

29.01 图15·复测既有方法

TTRL、单样本RL在Qwen上有效，在其他家族往往较弱；参照A.7中的在线伪标签、测试题训练和去熵项差异理解。

29.02 图16·机制边界

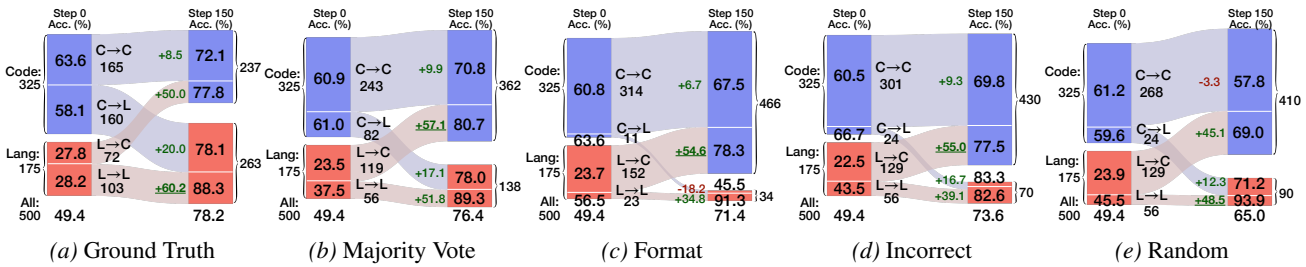
复合格式奖励可削弱Math收益，错误/真实奖励仍可能有效；抑制Bad-Code模型的代码则可能提高成绩。这支持代码是部分、依模型而异的机制。



(a) MATH-500 *(b)* AMC Avg. @8

(c) AIME 2024 Avg. @8 *(d)* AIME 2025 Avg. @8

30.01 *Figure 17.* We design a new type of reward, *no-repetition reward*, which assign a score of 1 to responses that do not contain obvious repetition and 0 to responses that contain obvious string repetition. We find, no-repetition reward effectively improves the performance of Qwen2.5-Math, while not others.



(a) Ground Truth *(b)* Majority Vote *(c)* Format *(d)* Incorrect *(e)* Random

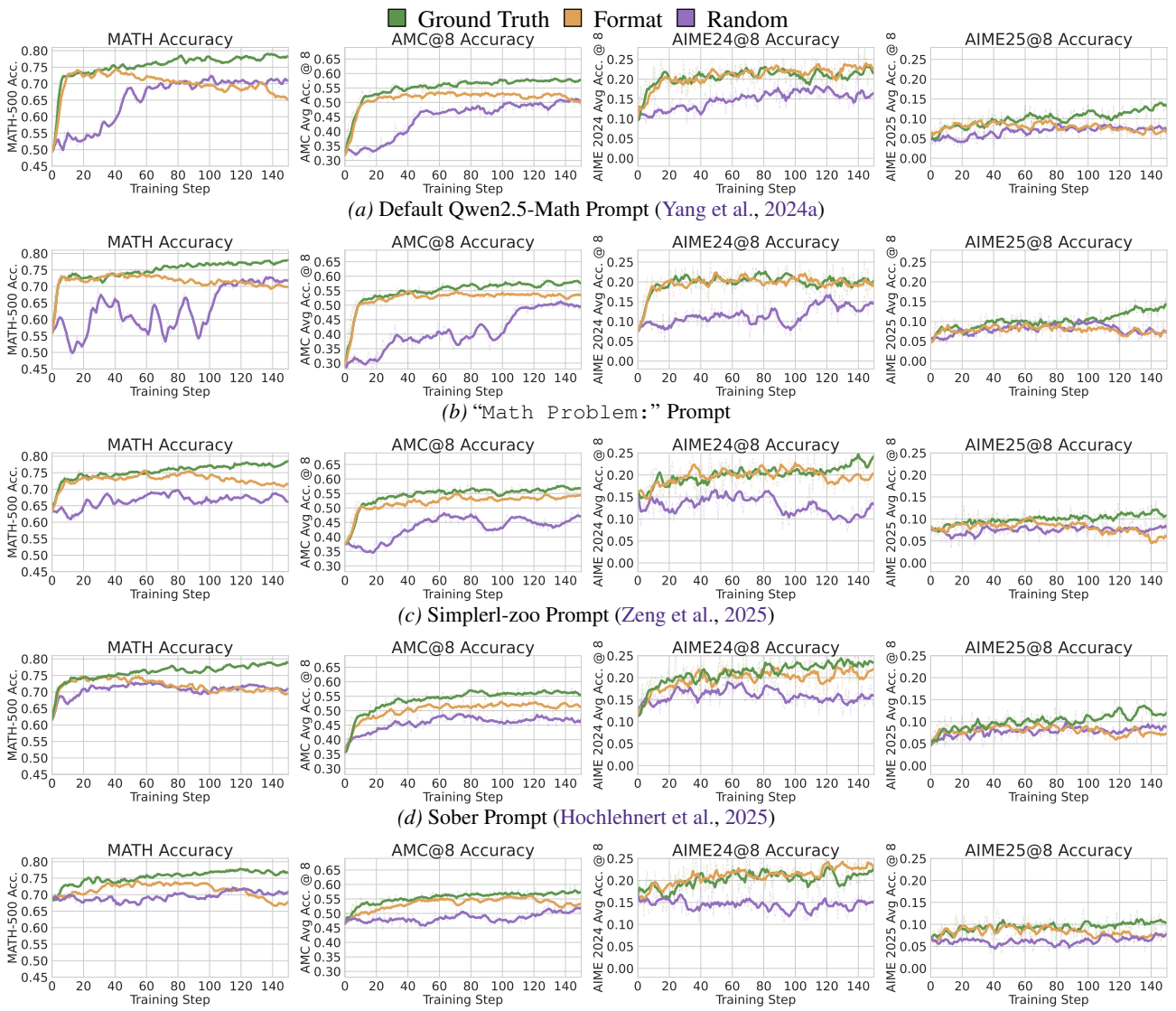
30.02 *Figure 18.* Reasoning strategy switching and fine-grained performance of Qwen2.5-Math-7B on the MATH-500 test set before and after RLVR with different training signals. Blue labels are the problem for which the model uses code reasoning, while red labels indicate reasoning traces using only natural language. Accuracy of each disjoint subset of problems before and after RLVR is shown in the shaded ends, and the size of each subset is shown in the lightly shaded region along with the change in accuracy. For all weak and spurious rewards, the model tends to use more code reasoning after RLVR. There is a small proportion of originally-code-reasoning problems being switched to language reasoning (Code→Lang); a majority of originally-language-reasoning problems convert to code reasoning (Lang→Code), on which we see the most significant performance increase after RLVR.

30.01 图17·重复抑制

仅减少明显重复就能提升Qwen-Math，说明无需数学标签也可通过改善输出行为涨分；其他家族不一定存在同样提升空间。

30.02 图18·流向与增益

带宽表示各策略转换组的题目数，端点为组内准确率。弱/虚假奖励下大量语言题转代码，转换组的准确率增长明显；真实奖励流向更复杂。



(a) Default Qwen2.5-Math Prompt (Yang et al., 2024a)

(b) "`Math Problem:`" Prompt

(c) Simplerl-zoo Prompt (Zeng et al., 2025)

(d) Sober Prompt (Hochlehnert et al., 2025)

(e) Spurious Prompt

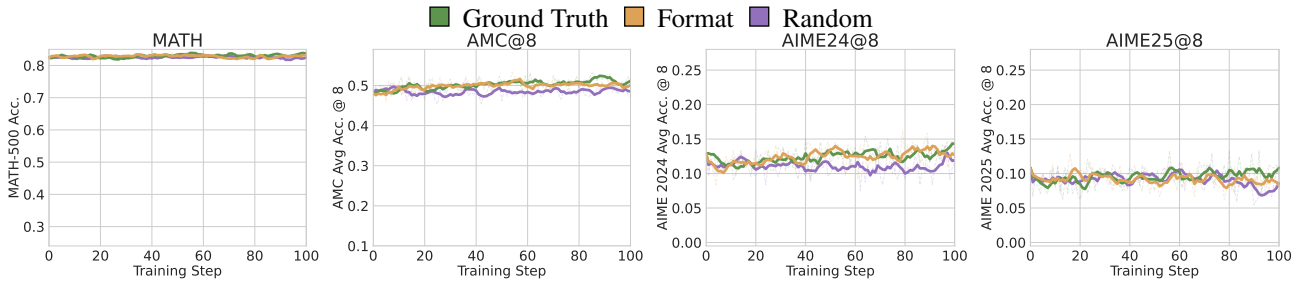
Figure 19. RLVR performance with different prompt templates on Qwen2.5-Math-7B. We show the choice of the prompt impacts both the before training performance and the RLVR training trajectories for Qwen2.5-Math-7B. We find the default prompt provided by Qwen2.5-Math (Yang et al., 2024a) results in lower initial performance; the other prompts offer higher initial performance, while Spurious Prompt offers the highest initial performance.

31.01 图19·提示×奖励

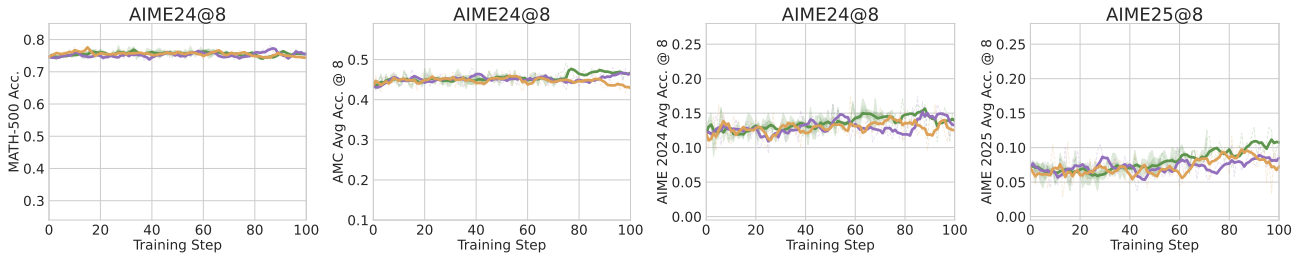
按行看提示、按列看任务，比较step0与后续轨迹。模板同时影响起点与训练动态，公平比较需统一模板及指标。

31.02 读图·控制起点

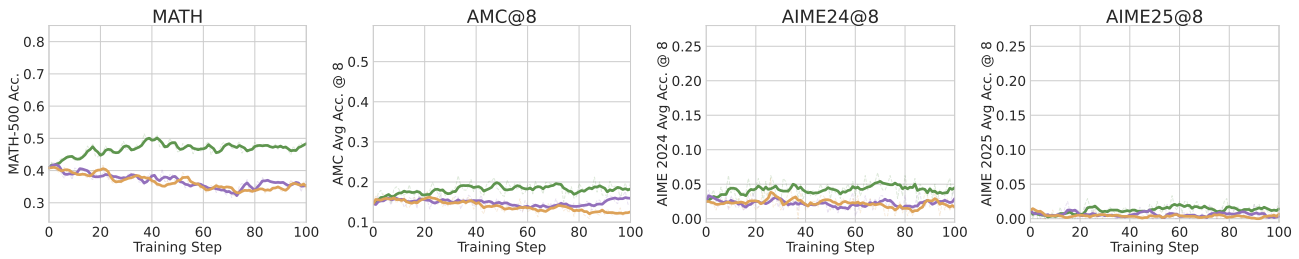
无关提示在此有最高起始成绩，未必拥有最大训练增量；应分别报告绝对成绩与相对起点的增益，不能混为同一效果。



(a) Qwen2.5-Math-7B-Instruct



(b) Qwen2.5-7B-Instruct Results



(c) Llama-3.1-Tulu-3-8B Results

32.01 *Figure 20.* RLVR performance on models that have undergone RL training. We find that Qwen Instruct models show minimal improvement from our RLVR, while Llama-3.1-Tulu-3-8B exhibits similar patterns to OLMo models (Figure 3). There is a clear performance distinction when training with ground truth labels for Llama-3.1-Tulu-3-8B, but not for the Qwen Instruct models.

32.01 图20·继续RL

Qwen指令模型增益小，Tulu3更能从真实监督受益。图中的平坦曲线仅说明当前数据与预算下的边际收益，不能证明永远无法再提升。

Original MATH Question:

Convert the point $(0, 3)$ in rectangular coordinates to polar coordinates. Enter your answer in the form (r, θ), where $r > 0$ and $0 \le \theta < 2\pi$.

Qwen2.5-Math-7B Solution (correct):

To convert the point $(0, 3)$ in rectangular coordinates to polar coordinates (r, θ), we need to use the following formulas:
1. The distance r from the origin to the point:

$$r = \sqrt{x^2 + y^2}$$

where $x = 0$ and $y = 3$.
2. The angle θ formed with the positive x-axis:
For the point $(0, 3)$:

- $r = \sqrt{0^2 + 3^2} = 3$
- Since $x = 0$ and $y = 3$, we are in the case where $x = 0$ and $y > 0$, so $\theta = \frac{\pi}{2}$.

Let's write the Python code to confirm these calculations and output the polar coordinates in the desired format.

```python
import math
# Rectangular coordinates
x = 0
y = 3
# Calculate r
r = math.sqrt(x**2 + y**2)
# Calculate θ
if x == 0 and y > 0:
    theta = math.pi / 2
elif x == 0 and y < 0:
    theta = 3 * math.pi / 2
else:
    theta = math.atan2(y, x)
# Ensure θ is in the range [0, 2π)
if theta < 0:
    theta += 2 * math.pi
print(f"r={r},θ = {theta}")
```

output: r = 3.0, θ = 1.5707963267948966
The polar coordinates for the point $(0, 3)$ are $(3, \frac{\pi}{2})$.
Thus, the final answer in the desired format is:
$\boxed{(3, \frac{\pi}{2})}$

Figure 21. Qualitative example of Qwen2.5-Math-7B outputs on one randomly picked question from MATH-500. Note that both the code and the corresponding code execution result are generated by Qwen2.5-Math-7B. **We do NOT provide any external code interpreter to the model.**

33.01 例21·任务

直角坐标(0,3)转极坐标，r>0且角度在[0,2π)，因此正确答案为(3,π/2)。

33.02 步骤·代码表示

模型先用几何/公式推导，再生成sqrt与角度分支代码；此例代码更多是已有解题结果的另一种表示。

33.03 输出·证据含义

1.5707963267948966是模型预测的文本，没有实际Python执行。这一成功例展示模拟能力，不能保证其他算例的同等精度。

34.01

Original MATH Question:

What is the distance, in units, between the points $(2, -6)$ and $(-4, 3)$? Express your answer in simplest radical form.

Qwen2.5-Math-7B Solution (correct):

To find the distance between two points (x_1, y_1) and (x_2, y_2) in a Cartesian plane, we use the distance formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Given the points $(2, -6)$ and $(-4, 3)$, we can substitute these values into the formula:

$$d = \sqrt{((-4) - 2)^2 + (3 - (-6))^2}$$

34.02

Let's break this down step-by-step and compute the result using Python.

```python
import math

# Coordinates of the points
x1, y1 = 2, -6
x2, y2 = -4, 3

# Calculate the differences
dx = x2 - x1
dy = y2 - y1

# Calculate the distance using the distance formula
distance = math.sqrt(dx**2 + dy**2)
print(distance)
```

34.03

output: 10.816653826391969
The distance between the points $(2, -6)$ and $(-4, 3)$ is approximately 10.82 units. To express the answer in simplest radical form, we need to check if this value can be simplified.
The distance formula calculation is:

$$d = \sqrt{((-4) - 2)^2 + (3 - (-6))^2} = \sqrt{(-6)^2 + (9)^2}$$
$$= \sqrt{36 + 81} = \sqrt{117} = \sqrt{9 \cdot 13} = 3\sqrt{13}$$

So, the exact distance in simplest radical form is $3\sqrt{13}$.
Thus, the final answer is:
$\boxed{3\sqrt{13}}$

Figure 22. Qualitative example of Qwen2.5-Math-7B outputs on one randomly picked question from MATH-500. Note that both the code and the corresponding code execution result are generated by Qwen2.5-Math-7B. **We do NOT provide any external code interpreter to the model.**

34.01 例22·任务

两点差为(-6,9)，距离√117=3√13；题目要求最简根式，而不是只给小数。

34.02 步骤·行为

生成Python计算浮点距离，再回到自然语言化简根式，展示代码与语言混合推理。

34.03 输出·核对

10.81665…与√117一致，最终根式也正确。代码与输出均由模型自生成，不能据此认定使用了工具。

35.01

Original MATH Question:

What is the base six equivalent of 999_{10}?

Owen2.5-Math-7B Solution (incorrect, ground truth: 4343_6):

35.02

To convert a decimal number to its base-six equivalent, we repeatedly divide the number by 6 and keep track of the remainders. The remainders, read in reverse order, give the base-six representation of the number. Here's the step-by-step process to convert 999_{10} to base six:

1. Divide the number by 6 and record the remainder.
2. Update the number to the quotient of the division.
3. Repeat steps 1 and 2 until the quotient is 0.
4. The base-six representation is the sequence of remainders read from bottom to top.

Let's implement this in Python to get the accurate result:

```python
def decimal_to_base_six(n):
    if n == 0:
        return "0"

    base_six_digits = []
    while n > 0:
        remainder = n % 6
        base_six_digits.append(str(remainder))
        n = n // 6

    # The base-six representation is the reverse of the collected remainders
    base_six_digits.reverse()
    return "".join(base_six_digits)

# Convert 999 from decimal to base six
decimal_number = 999
base_six_representation = decimal_to_base_six(decimal_number)
print(base_six_representation)
```

35.03

4323

The base-six equivalent of 999_{10} is $\boxed{\mathbf{4323}}$.

Figure 23. Qualitative example of Qwen2.5-Math-7B outputs on one randomly picked question from MATH-500. Note that both the code and the corresponding code execution result are generated by Qwen2.5-Math-7B. **We do NOT provide any external code interpreter to the model.**

35.01 例23·任务

999十进制转六进制，标准答案4343；反算 4×216+3×36+4×6+3=999。

35.02 步骤·算法

反复除6取余、最后逆序，是正确进制转换算法；代码思路与后面的“输出”正确性必须分开检查。

35.03 错误定位

模型声称代码输出4323，但它对应987而不是999。若实际运行所列转换函数应得到4343，显示模拟执行结果可能幻觉。

Numerically Perturbed MATH Question:

What is the base six equivalent of 1000_{10}?

Qwen2.5-Math-7B Solution (correct):

To convert a decimal number (base 10) to a base 6 number, we repeatedly divide the number by 6 and keep track of the remainders. The base 6 number is then composed of the remainders read in reverse order. Let's perform this conversion step by step.
1. Divide the number by 6, record the remainder, and replace the number with the quotient. 2. Repeat until the quotient is 0. 3. The base 6 number is the sequence of remainders read in reverse order.
Let's do this process using Python code.

```python
def decimal_to_base_six(n):
    if n == 0:
        return '0'

    base_six_digits = []
    while n > 0:
        remainder = n % 6
        base_six_digits.append(str(remainder))
        n = n // 6

    # The remainders give the base 6 number in reverse order
    base_six_digits.reverse()
    return ''.join(base_six_digits)

# Convert 1000 from base 10 to base 6
base_six_number = decimal_to_base_six(1000)
print(base_six_number)
```

4344
The base six equivalent of 1000_{10} is $\boxed{4344_6}$.

Figure 24. Qualitative example of Qwen2.5-Math-7B outputs on the numerically perturbed MATH-500 question as shown in Figure 23. Note that both the code and the corresponding code execution result are generated by Qwen2.5-Math-7B. **We do NOT provide any external code interpreter to the model.**

36.01 例24·数字扰动

把999改为1000，题型不变；正确六进制为4344。用于观察策略是否依赖原始数值。

36.02 步骤·策略保留

仍用除基取余代码，模型这次生成正确结果。邻近数值答对不等于原题答错只是算法不会，也不能证明完全没有记忆影响。

36.03 核对

4×216+3×36+4×6+4=1000，结果正确；它依然是生成的输出，没有外部执行器。

Numerically Perturbed MATH Question:

What is the base six equivalent of 998_{10}?

Qwen2.5-Math-7B Solution (correct):

To convert the decimal number 998_{10} to its base six equivalent, we need to repeatedly divide the number by 6 and keep track of the remainders. The base six digits are the remainders read from bottom to top (from last division to first).
Here is the step-by-step process:

1. Divide the number by 6.
2. Record the remainder.
3. Update the number to the quotient of the division.
4. Repeat steps 1-3 until the quotient is 0.
5. The base six number is the sequence of remainders read from bottom to top.

Let's do this step-by-step for 998_{10}:

$$998 \div 6 = 166 \text{ remainder } 2 \tag{4}$$
$$166 \div 6 = 27 \text{ remainder } 4 \tag{5}$$
$$27 \div 6 = 4 \text{ remainder } 3 \tag{6}$$
$$4 \div 6 = 0 \text{ remainder } 4 \tag{7}$$

The remainders, read from bottom to top, give us the base six number. So, 998_{10} in base six is 4342_6.
We can verify this by converting back to decimal:

$$4 \cdot 6^3 + 3 \cdot 6^2 + 4 \cdot 6^1 + 2 \cdot 6^0 = 4 \cdot 216 + 3 \cdot 36 + 4 \cdot 6 + 2 \tag{8}$$
$$= 864 + 108 + 24 + 2 \tag{9}$$
$$= 998_{10} \tag{10}$$

Now, let's implement this in Python to ensure the accuracy.

```
def decimal_to_base_six(n):
    if n == 0:
        return "0"
    base_six_digits = []
    while n > 0:
        remainder = n % 6
        base_six_digits.append(str(remainder))
        n = n // 6
    base_six_digits.reverse()
    return ''.join(base_six_digits)

# Convert 998 from base 10 to base 6
base_six_number = decimal_to_base_six(998)
print(base_six_number)
```

```
4342
```

The base six equivalent of 998_{10} is $\boxed{4342_6}$.

Figure 25. Qualitative example of Qwen2.5-Math-7B outputs on the numerically perturbed MATH-500 question as shown in Figure 23. Note that both the code and the corresponding code execution result are generated by Qwen2.5-Math-7B. **We do NOT provide any external code interpreter to the model.**

37.01 例25·数字扰动

998转六进制，正确结果4342。此页先展示自然语言计算，再写代码。

37.02 步骤·可验证计算

998÷6余2，166÷6余4，27÷6余3，4÷6余4；逆序得到4342，反算也一致。

37.03 代码·作用

同样的转换函数与前述手算一致；这里能直接核对手算链，正确性不需要依赖模型生成的执行日志。

蓝色数字对应段落 / 图表；术语在首次出现处批注

Numerically Perturbed MATH Question:

What is the base six equivalent of 100_{10}?

Qwen2.5-Math-7B Solution (correct):

To convert a decimal number (base 10) to a base-six number, we can repeatedly divide the number by 6 and keep track of the remainders. The conversion process involves the following steps:

1. Divide the number by 6.
2. Record the remainder.
3. Update the number to the quotient from the division.
4. Repeat the process until the quotient is 0.
5. The base-six number is the sequence of remainders read in reverse order.

Let's convert 100_{10} to base six using Python.

```
def decimal_to_base_six(n):
    if n == 0:
        return "0"

    base_six_digits = []

    while n > 0:
        remainder = n % 6
        base_six_digits.append(str(remainder))
        n = n // 6

    # The base-six number is the sequence of remainders read
    # in reverse order
    base_six_digits.reverse()
    return ''.join(base_six_digits)

# Convert 100 (decimal) to base six
decimal_number = 100
base_six_number = decimal_to_base_six(decimal_number)
print(base_six_number)
```

```
244
```

The base six equivalent of 100_{10} is $\boxed{244_6}$.

Figure 26. Qualitative example of Qwen2.5-Math-7B outputs on the numerically perturbed MATH-500 question as shown in Figure 23. Note that both the code and the corresponding code execution result are generated by Qwen2.5-Math-7B. **We do NOT provide any external code interpreter to the model.**

38.01 例26·更大数值变化

100转六进制为244，较999变化更大，模型仍保持代码解法。

38.02 步骤·算法概括

循环除6、保存余数、反转列表，复用了同一通用模板；此案例支持代码策略在若干数值扰动下的稳定性。

38.03 核对

2×36+4×6+4=100，结果正确。少量成功例只能支持局部观察，不能量化整体鲁棒率。

39.01

Rephrased MATH Question in an Alternative Narrative:

If I have 999 objects in standard decimal counting, and I need to express this quantity using only the digits 0-5 in a positional numeral system, what sequence of digits would represent this amount?

Qwen2.5-Math-7B Solution 2 (correct):

39.02

To represent 999 using only the digits 0-5, we need to convert it to a base 6 number system. We need to divide this number by the powers of 6 and take the remainder until we run out of powers of 6.
999 divided by 6^3 (which is 216) is 4 with a remainder of 135.
135 divided by 6^2 (which is 36) is 3 with a remainder of 27.
27 divided by 6^1 (which is 6) is 4 with a remainder of 3.
3 divided by 6^0 (which is 1) is 3 with no remainder.
So, 999 in base 6 is 4343_6.

$$\begin{aligned} 4 \times 6^3 + 3 \times 6^2 + 4 \times 6^1 + 3 \times 6^0 &= 4 \times 216 + 3 \times 36 + 4 \times 6 + 3 \times 1 \\ &= 864 + 108 + 24 + 3 \\ &= 999 \end{aligned}$$

39.03

Therefore, the representation of 999 using only digits 0-5 in a base-6 numeral system is 4343_6.

Figure 27. Qualitative example of Qwen2.5-Math-7B outputs on the MATH-500 question shown in Figure 23 but with an alternative narrative. Qwen2.5-Math-7B uses natural language to answer the rephrased question without code reasoning.

39.01 例27·只换叙述

把999进制题改成“只能用0至5表示物体数量”，数学目标未变，模型改用纯自然语言。

39.02 步骤·语言策略

按6的幂贪心取系数，依次得到4、3、4、3；与原题的代码策略不同。

39.03 发现

重述后答案4343正确，原始表述的代码输出反而错误。策略改变与正确性变化相关，但单例不足以证明哪种表示普遍更好。

40.01

Rephrased MATH Question in an Alternative Narrative:

If I have 998 objects in standard decimal counting, and I need to express this quantity using only the digits 0-5 in a positional numeral system, what sequence of digits would represent this amount?

Qwen2.5-Math-7B Solution 2 (correct):

40.02

To express the quantity of 998 using only the digits 0-5 in a positional numeral system, we need to convert 998 to a base-6 number. Here's the step-by-step process:
1. **Determine the largest power of 6 less than or equal to 998:** - $6^0 = 1$ - $6^1 = 6$ - $6^2 = 36$ - $6^3 = 216$ - $6^4 = 1296$ (which is greater than 998)
So, the largest power of 6 less than or equal to 998 is $6^3 = 216$.
2. **Find the coefficient for 6^3:** - $998 \div 216 \approx 4.62$ - The integer part is 4, so the coefficient for 6^3 is 4. - Subtract $4 \times 216 = 864$ from 998 to get the remainder: $998 - 864 = 134$.
3. **Find the coefficient for 6^2:** - $134 \div 36 \approx 3.72$ - The integer part is 3, so the coefficient for 6^2 is 3. - Subtract $3 \times 36 = 108$ from 134 to get the remainder: $134 - 108 = 26$.
4. **Find the coefficient for 6^1:** - $26 \div 6 \approx 4.33$ - The integer part is 4, so the coefficient for 6^1 is 4. - Subtract $4 \times 6 = 24$ from 26 to get the remainder: $26 - 24 = 2$.
5. **Find the coefficient for 6^0:** - The remainder is 2, so the coefficient for 6^0 is 2.

40.03

Putting it all together, the base-6 representation of 998 is 4342_6.
Therefore, the sequence of digits that represents 998 in a base-6 positional numeral system is $\boxed{4342}$.

Figure 28. Qualitative examples of Qwen2.5-Math-7B outputs on the MATH-500 question shown in Figure 23 but with an alternative narrative. Qwen2.5-Math-7B uses natural language to answer the rephrased question without code reasoning.

40.01 例28·重述与数字扰动

998的问题换成物体数量叙述，同样改用自然语言推导，答案4342。

40.02 步骤·分解

先找6^3=216，再分配系数4、3、4、2，余数递减。原页较密的列表仍可按这个主线阅读。

40.03 发现

数字扰动版与改写版都正确，但表示方式不同。支持题目措辞会影响策略选择，而不只是数学内容决定策略。

41.01

Rephrased MATH Question in an Alternative Narrative:

If I have 100 objects in standard decimal counting, and I need to express this quantity using only the digits 0-5 in a positional numeral system, what sequence of digits would represent this amount?

Qwen2.5-Math-7B Solution 1 (incorrect, ground truth: 244):

41.02

To represent the number 100 using only the digits 0-5, you can use a base-6 numeral system. In a base-6 system, each digit represents a power of 6, starting from the right with 6^0. The number 100 in base-10 can be converted to base-6 as follows:

$$\begin{aligned} 100 &= 2 \times 6^2 + 0 \times 6^1 + 4 \times 6^0 \\ &= 2 \times 36 + 0 \times 6 + 4 \times 1 \\ &= 72 + 0 + 4 \\ &= 76 \end{aligned}$$

41.03

However, since we are using base-6, the correct representation of 100 in this system is 204 ($2 \times 6^2 + 0 \times 6^1 + 4 \times 6^0$).

Figure 29. Qualitative examples of Qwen2.5-Math-7B outputs on the MATH-500 question shown in Figure 23 but with an alternative narrative and numerical perturbation. Qwen2.5-Math-7B uses natural language to answer the rephrased question without code reasoning. The equation expansion step in the solution is incorrect, leading to a final wrong answer: the calculation claims that $100 = 2 \times 6^2 + 0 \times 6^1 + 4 \times 6^0 = 72 + 0 + 4 = 76$, which is mathematically inconsistent.

41.01 例29·反例

100的重述版换用语言推理，却输出204。原先代码版给出正确244，说明改写既可纠错也可引入错误。

41.02 错误·位置

模型选的系数2、0、4实际表示76，却把它等同100；真正分解应为2×36+4×6+4=100。

41.03 精读发现

最后一句没有修复中间矛盾，只坚持错误204。此例提醒：流畅解释和自称正确不能替代可验证计算。